OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 16

COMPUTER ARCHITECTURE & HARDWARE ORGANIZATION

SUGGESTED TIME: 4 PERIODS

IN THIS CHAPTER

This chapter takes the learner inside the machine: from the von Neumann architecture and hardware organisation of a computer to instruction sets, assembly language and the fetch-decode-execute cycle. Through two hands-on C projects, students simulate a CPU and manipulate registers, turning abstract architecture into concrete, examinable skills.

- By the end of this chapter I can describe the von Neumann architecture and the role of each of its components (CPU, memory, I/O, buses).
- By the end of this chapter I can identify and explain the function of the main CPU registers (PC, IR, MAR, MDR, ACC, flags).
- By the end of this chapter I can explain the fetch-decode-execute cycle step by step with timing.
- By the end of this chapter I can distinguish machine code, assembly language and high-level languages, and explain the role of assemblers, compilers and interpreters.
- By the end of this chapter I can read and trace short assembly programs involving registers, memory and simple arithmetic.
- By the end of this chapter I can write a C program that simulates main memory and the instruction cycle of a simple CPU.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. Section 1: Computer Architecture & Hardware Organization (Lesson 89)
2. Section 2: Instruction Set Architecture & Low-Level Processing (Lesson 90)
3. Section 3: Hands-on C Project — CPU Instruction Cycle & Memory Simulation (Lesson 91)
4. Section 4: Hands-on C Project — Assembly Code & Register Manipulation Lab (Lesson 92)
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can describe the von Neumann architecture and the role of each of its components (CPU, memory, I/O, buses).
- By the end of this chapter I can identify and explain the function of the main CPU registers (PC, IR, MAR, MDR, ACC, flags).
- By the end of this chapter I can explain the fetch-decode-execute cycle step by step with timing.
- By the end of this chapter I can distinguish machine code, assembly language and high-level languages, and explain the role of assemblers, compilers and interpreters.
- By the end of this chapter I can read and trace short assembly programs involving registers, memory and simple arithmetic.
- By the end of this chapter I can write a C program that simulates main memory and the instruction cycle of a simple CPU.
- By the end of this chapter I can use bitwise operators in C to manipulate register contents at bit level.
- By the end of this chapter I can compare addressing modes and instruction formats and evaluate simple performance factors (clock speed, cache, word size).

PREREQUISITES

- Basic C programming: variables, arrays, functions, loops (Lower Sixth and earlier Upper Sixth chapters)
- Binary and hexadecimal number systems and conversions
- Boolean logic and logic gates (AND, OR, NOT, XOR)
- General structure of a computer system: hardware vs software (Form 5 / Lower Sixth)

1 SECTION 1: COMPUTER ARCHITECTURE & HARDWARE ORGANIZATION (LESSON 89)

1.1 Opening the Black Box

Imagine a busy cybercafé in Buea. The owner has just upgraded the machines: same processor speed advertised on the box, but the new ones have more cache memory. Customers notice that web pages load snappier, video editing renders faster, and the old "lag" when switching between applications has vanished. **Why does more cache make a difference if the clock speed is identical?** Why do some C programs finish in milliseconds while others with the same algorithm crawl? The answer lies hidden inside the machine: in the way the processor fetches, decodes and executes instructions, and in how registers, buses and memory cooperate. In this section you will open the "black box" of the computer and learn the vocabulary and concepts that GCE examiners test every year.

1.2 Computer Architecture vs. Hardware Organisation

DEFINITION **Computer Architecture**

The **computer architecture** is the functional specification of a computer as seen by the programmer: the instruction set, the number and size of registers, the memory addressing modes, and the interrupt structure. It answers the question "What does the machine do?"

DEFINITION **Hardware Organisation**

The **hardware organisation** (or microarchitecture) is the physical implementation of that architecture: the actual datapaths, control signals, pipeline stages, cache hierarchy, and bus timing. It answers "How is it built?"

KEY POINTS **Key Distinction**

- Two processors can share the same architecture (e.g., both implement the ARMv8 instruction set) but have different organisations (one has a 3-stage pipeline and 32 KB L1 cache; the other has a 10-stage pipeline and 64 KB L1 cache).
- The programmer sees the architecture; the chip designer builds the organisation.

1.3 The Von Neumann Stored-Program Concept

Before 1945, “programming” meant rewiring plugboards. The **von Neumann architecture** introduced a revolutionary idea:

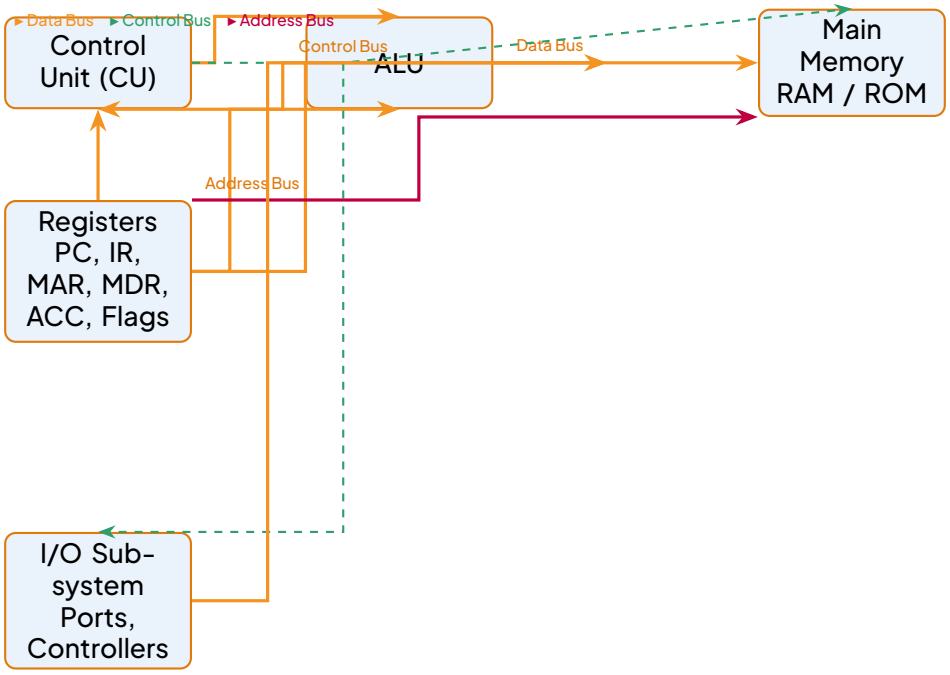
DEFINITION **Stored-Program Concept**

Instructions and data reside together in the same memory and are fetched sequentially (unless a branch instruction modifies the flow). The processor does not distinguish between an instruction bit-pattern and a data bit-pattern until it interprets it.

PROPERTY **Von Neumann Model – Five Functional Units**

Every von Neumann machine comprises:

1. Memory – stores both instructions and data, addressed linearly.
2. Arithmetic-Logic Unit (ALU) – performs arithmetic and logical operations.
3. Control Unit (CU) – generates the control signals that orchestrate fetch, decode, execute.
4. Input/Output (I/O) – communicates with the outside world.
5. Buses – shared communication pathways (data, address, control) linking the units.



Von Neumann architecture: CPU (CU, ALU, registers), memory, I/O and the three system buses.

EXEMPLE **Instruction vs. Data in Memory**

Suppose memory location `0x1000` contains the 16-bit pattern `0010 0001 0000 0011`.

- If the PC holds `0x1000`, the CU treats it as an instruction (e.g., `LOAD ACC, 0x0003`).
- If an instruction says `LOAD ACC, [0x1000]`, the same pattern is treated as data (the value `0x2103` = 8451 decimal).

The bit-pattern is identical; only the context (fetch vs. operand fetch) differs.

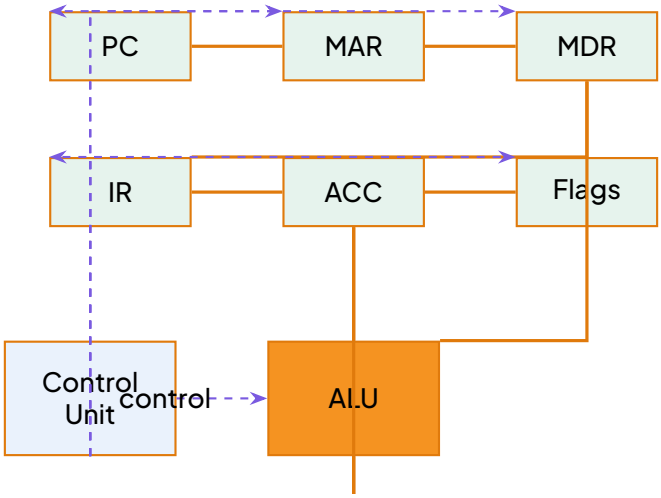
1.4 Inside the CPU: Control Unit and ALU

The **Control Unit (CU)** is the "conductor": it reads the instruction from the **Instruction Register (IR)**, decodes it, and emits the precise sequence of control signals (e.g., `ALU_OP=ADD`, `MAR_LOAD=1`, `MEM_READ=1`) that drive the datapath.

The **Arithmetic-Logic Unit (ALU)** is the "calculator": it receives operands from registers (often the **Accumulator (ACC)** and a temporary register), performs the requested operation (ADD, SUB, AND, OR, SHIFT, COMPARE), and deposits the result back into a register while updating the **Status/Flag Register** (Zero, Carry, Negative, Overflow).

METHOD **How the CU and ALU Cooperate – One Instruction Cycle**

1. Fetch: CU places PC on address bus → memory returns instruction into MDR → MDR → IR → PC incremented.
2. Decode: CU examines IR opcode → determines operation, addressing mode, operand location.
3. Execute: CU asserts control signals; ALU computes; result written to register/memory; flags updated.
4. Interrupt Check: CU checks interrupt line; if active, saves PC and vectors to handler.



Internal CPU structure: registers, ALU and CU connected by an internal data bus (solid) and control lines (dashed purple).

1.5 Main Registers – The CPU's "Workbench"

Register	Role
Program Counter (PC)	Holds the address of the **next** instruction to fetch. Auto-increments after fetch (unless a branch loads a new value).
Instruction Register (IR)	Holds the **current** instruction after fetch. Split into op-code (CU) and operand field (address/immediate).
Memory Address Register (MAR)	Holds the address for **any** memory access (instruction fetch or data read/write). Connected directly to the address bus.
Memory Data Register (MDR)	Holds the **data** being transferred to/from memory. Connected to the data bus.
Accumulator (ACC)	Primary operand/result register for ALU operations. In accumulator-based ISAs, one operand is implicitly ACC.
Status / Flag Register	Individual bits set by ALU: **Z** (zero), **C** (carry), **N** (negative), **V** (overflow). Used by conditional branches.

WARNING **Accumulator ≠ Main Memory**

Examiner trap: "The accumulator stores the program." False. The accumulator is a single CPU register (typically 16–64 bits) holding one operand/result at a time. The program resides in main memory (RAM/ROM). Confusing the two loses marks every session.

1.6 Memory Organisation

DEFINITION **Memory Address and Word**

- Address: An unsigned integer (0 to $2^n - 1$) identifying a unique location. n = address bus width.
- Word: The natural unit of data the processor handles (e.g., 8, 16, 32, 64 bits). Word size often equals data bus width.

PROPERTY **Address Bus Width → Maximum Addressable Memory**

If the address bus has n lines, the processor can address 2^n distinct locations.

$$\text{Max memory (bytes)} = 2^n \times \frac{\text{word size (bits)}}{8}$$

Example: 16-bit address bus, 8-bit word → 2^{16} = 65 536 bytes = 64 KB. 32-bit address bus, 32-bit word → $2^{32} \times 4$ = 16 GB.

WORKED EXAMPLE 1 **Address Bus Calculation**

Statement: A processor has a 20-bit address bus and a 16-bit data bus. What is the maximum addressable memory in (a) bytes, (b) kilobytes, (c) megabytes?

SOLUTION

Solution:

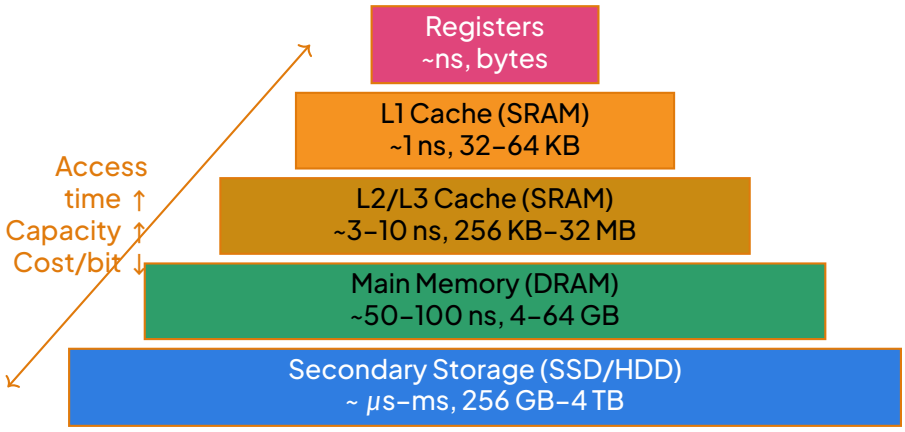
$$\text{Number of addresses} = 2^{20} = 1\,048\,576$$
$$\text{Word size} = 16 \text{ bits} = 2 \text{ bytes}$$
$$\text{(a) Max bytes} = 2^{20} \times 2 = 2\,097\,152 \text{ bytes}$$
$$\text{(b) KB} = 2\,097\,152 / 1024 = 2\,048 \text{ KB}$$
$$\text{(c) MB} = 2\,048 / 1024 = 2 \text{ MB}$$

Answer: 2 MB (classic 8086 real-mode limit).

1.7 RAM vs. ROM

Feature	RAM (Random Access Memory)	ROM (Read-Only Memory)
Volatility	Volatile (loses content on power-off)	Non-volatile (retains content)
Write access	Read **and** write	Traditionally read-only (mask ROM); modern: Flash/EEPROM allow limited writes
Speed	Fast (nanoseconds)	Comparable for reads
Typical use	Programs + data during execution	Boot firmware (BIOS/UEFI), embedded firmware

1.8 Memory Hierarchy – Speed, Size, Cost Trade-off



Memory hierarchy pyramid: as we go down, capacity increases and cost per bit falls, but access time grows. Registers and caches are SRAM; main memory is DRAM; secondary is non-volatile.

KEY POINTS **Why Hierarchy Works – Locality of Reference**

- Temporal locality: Recently accessed items are likely accessed again soon → keep in cache.
- Spatial locality: Items near a recently accessed item are likely accessed next → fetch blocks (cache lines) not single words.

A program with good locality (e.g., sequential array scan) runs much faster on a cached machine than one with random access patterns.

1.9 Buses – The Communication Highways

Bus	Direction	Function
Data Bus	Bidirectional	Carries instruction codes, operands, results between CPU, memory, I/O. Width = word size typically.
Address Bus	Unidirectional (CPU → memory/I/O)	Carries the address of the location to access. Width determines max addressable memory.
Control Bus	Mixed	Carries synchronisation and command signals: `MEM_READ`, `MEM_WRITE`, `IO_READ`, `IO_WRITE`, `INTERRUPT_ACK`, `CLOCK`, `RESET`.

PROPERTY **Bus Width and Performance**

- Data bus width: Determines how many bits transfer per cycle. A 64-bit bus moves 8 bytes/cycle vs. 4 bytes for 32-bit → double bandwidth for same clock.
- Address bus width: Sets the physical address space ceiling (see property above).
- BUS CONTENTION: Only one device may drive the bus at a time; arbitration (priority, daisy-chain, distributed) decides who wins.

1.10 Input/Output Subsystem – Overview

- **I/O Ports:** Addressable registers on device controllers (e.g., keyboard status port at 0×60, data port at 0×61).
- **Device Controllers:** Hardware that manages a specific device (disk controller, GPU, UART). They have their own registers, buffers, and often DMA (Direct Memory Access) engines.
- **Interrupts (overview):** A device asserts `INT` line → CU finishes current instruction, saves PC, loads PC with **interrupt vector** address → runs **Interrupt Service Routine (ISR)** → returns via `RETI`. This avoids wasteful polling.

DID YOU KNOW? **Cameroon Context – Mobile Money Servers**

MTN Mobile Money and Orange Money platforms in Cameroon run on servers with large L3 caches (64+ MB) and NVMe SSDs. The database workload has high temporal locality (recent transactions re-read for balance checks); cache hits avoid millisecond DRAM/SSD latency, enabling thousands of transactions per second.

1.11 Performance Factors – Beyond Clock Speed

KEY POINTS **GCE-Useful Performance Factors**

1. Clock speed (frequency): Cycles per second (GHz). Higher → more cycles, but not linearly faster if CPI (cycles per instruction) rises.
2. Word size: Wider data path → more bits processed per instruction (e.g., 64-bit ADD vs. two 32-bit ADDs for 64-bit integers).
3. Cache size & levels: Larger L1/L2/L3 → higher hit rate → fewer stalls waiting for DRAM.
4. Number of cores: True parallelism for multi-threaded workloads. Not a simple multiplier (Amdahl's law, memory bandwidth contention).

5. Pipelining / Superscalar / Out-of-order: Microarchitectural techniques that reduce average CPI.

WARNING **Clock Speed Alone is Misleading**

A 3.0 GHz Pentium 4 (deep pipeline, high CPI on branch mispredicts) can be slower than a 2.4 GHz Core i5 (shorter pipeline, better branch prediction, larger cache) on real workloads. Examiners expect you to cite architecture, cache, word size, cores – not just GHz.

1.12 CISC vs. RISC – Brief Comparative Note

Aspect	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Examples	x86 (Intel/AMD), VAX, 68k	ARM, MIPS, RISC-V, PowerPC
Instruction count	Large (100–300+)	Small (30–50)
Instruction length	Variable (1–15 bytes)	Fixed (usually 32 bits)
Addressing modes	Many (10+)	Few (3–5: register, immediate, PC-relative, load/store)
Memory access	Many instructions access memory	**Load/Store only** (ALU ops use registers only)
Microcode	Heavy (complex instructions decoded to micro-ops)	Minimal / hardwired control
Registers	Few (8–16 general purpose)	Many (16–32 general purpose)
Pipelining	Harder (variable length, side effects)	Easy (fixed length, regular)
Code density	Higher (complex instructions do more)	Lower (more instructions per task)

EXEMPLE **CISC vs. RISC in One Line**

CISC: `MULT [mem], REG` (single instruction, microcoded, variable cycles).
RISC: `LOAD R1, [mem] ; MUL R2, R1, R3 ; STORE [mem], R2` (three simple, fixed-cycle instructions).

METHOD **Identifying a Component's Role in a Diagram**

When faced with an unlabelled block diagram in the exam:

1. Stores bits? → Memory or Register.
2. Transforms bits (add, and, shift)? → ALU.
3. Generates control signals from opcode? → Control Unit.
4. Carries signals between blocks? → Bus (data/address/control by width and direction).

1.13 Worked Examination-Style Question

WORKED EXAMPLE 2 **GCE-Style Question**

Statement: A processor has a 24-bit address bus and a 32-bit data bus. It uses a two-level cache: L1 = 32 KB, L2 = 256 KB. Main memory is 512 MB DRAM. The clock is 2.0 GHz.

1. Calculate the maximum physical address space in GB.
2. Explain why the L2 cache is larger but slower than L1.
3. The manufacturer releases a new version with a 3.0 GHz clock but the same cache sizes. A benchmark shows only 15% speedup. Give two microarchitectural reasons (other than cache) why the speedup is not 50%.

SOLUTION

Solution:

1. Address lines = 24 → 2^{24} addresses. Data bus = 32 bits = 4 bytes.
   Max space = $2^{24} \times 4$ bytes = 2^{26} bytes = 64 MB. (Not GB – 24 bits only gives 64 MB!)
2. L1 uses smaller, faster SRAM cells close to the core (1–2 cycles). L2 uses larger, denser SRAM cells further away (5–10 cycles). Capacity/latency trade-off.
3. Reasons: (a) Memory wall: DRAM latency (~60 ns) ≈ 120 cycles at 2 GHz → 180 cycles at 3 GHz; CPU spends more cycles waiting. (b) Branch misprediction penalty: Deeper pipeline at higher clock → more cycles lost per mispredict. (c) Bus contention / bandwidth saturation: Same bus width, more requests per second → queueing delays.

1.14 Summary Checklist for Lesson 89

KEY POINTS **Must-Know for the Exam**

- Define architecture vs. organisation.

- State the stored-program concept precisely.
- Draw and label the von Neumann block diagram (CPU, memory, I/O, three buses).
- Name and explain PC, IR, MAR, MDR, ACC, Flags.
- Derive max memory from address bus width (2^n × word size).
- Contrast RAM vs. ROM, SRAM vs. DRAM.
- Sketch the memory hierarchy pyramid with typical sizes/times.
- Explain data, address, control buses (direction, width effect).
- List performance factors beyond clock speed.
- Compare CISC vs. RISC in a table (at least 5 rows).

EXERCISE 1 **Lesson 89 Practice**

1. A 16-bit microprocessor has a 20-bit address bus. What is the maximum addressable memory in KB?
2. During the fetch phase, which register drives the address bus and which register receives the instruction from the data bus?
3. Why is the accumulator not part of main memory? Give a one-sentence distinction.
4. A program runs on a machine with 32 KB L1 cache. The same program runs slower on a machine with 8 KB L1 cache but identical clock and core. Explain using locality of reference.
5. Complete the table: CISC instructions are _____ length; RISC instructions are _____ length. CISC uses _____ for complex instructions; RISC uses _____ control.

SOLUTION **Lesson 89 Practice**

1. 2^{20} addresses × 2 bytes (16-bit word) = 2 097 152 bytes = 2048 KB = 2 MB.
2. MAR drives the address bus; MDR receives from data bus, then transfers to IR.
3. The accumulator is a single CPU register holding one operand/result; main memory is a large addressable array holding the program and data.
4. The 32 KB L1 captures more of the program's working set (temporal/spatial locality),

yielding a higher hit rate and fewer DRAM stalls.

5. Variable / fixed; microcode / hardwired.

2 SECTION 2: INSTRUCTION SET ARCHITECTURE & LOW-LEVEL PROCESSING (LESSON 90)

In Section 1 we opened the computer case and studied the hardware: the CPU with its control unit, ALU and registers, the memory, and the buses that connect them. But hardware alone is inert. A processor is like a brilliant but extremely literal-minded worker: it can only obey a fixed, limited vocabulary of orders. This section studies that vocabulary — the **Instruction Set Architecture** — and the precise way the CPU fetches, decodes and executes each order. This is the heart of low-level processing, and a favourite topic of GCE Advanced Level examiners.

2.1 Instruction Set Architecture and Instruction Format

Imagine buying two different brands of calculator. Both can add, subtract, multiply and divide, but the exact buttons and the order in which you press them differ. In the same way, every processor family (Intel x86, ARM, MIPS…) understands its own specific set of machine orders. The complete description of those orders is called the Instruction Set Architecture.

DEFINITION **Instruction Set Architecture (ISA)**

The **Instruction Set Architecture** (ISA) of a processor is the complete set of machine instructions that the processor can execute, together with the format of each instruction, the registers available to the programmer, and the addressing modes used to locate data. The ISA is the contract between hardware and software: it is everything a programmer (or a compiler) needs to know to write machine-level programs for that processor.

A **machine instruction** is simply a binary number, stored in memory, that the control unit interprets as an order. Every instruction is divided into two parts:

DEFINITION **Opcode and Operand**

The **opcode** (operation code) is the part of an instruction that specifies which operation must be performed (add, move, jump…). The **operand** part specifies the data to be used, or where to find it (a register, a memory address, or a constant value).

For example, in a simple 16-bit machine we might reserve the first 4 bits for the opcode and the remaining 12 bits for the operand:



Instruction format of a simple 16-bit machine: 4-bit opcode, 12-bit operand.

With 4 bits of opcode we can encode $2^4 = 16$ different operations, and with 12 bits of operand we can address $2^{12} = 4096$ memory cells. Real processors use longer words and several formats, but the principle is identical.

2.2 Types of Instructions

Although ISAs differ, virtually all of them provide four families of instructions.

Family	Examples	Purpose
Data transfer	LOAD, STORE, MOV	Move data between memory and registers, or between registers, without changing it.
Arithmetic / logic	ADD, SUB, MUL, AND, OR, NOT	Perform calculations in the ALU and logical (bit-by-bit) operations.
Control flow	JMP, JZ, JNZ, CALL, RET	Alter the order of execution: jumps, conditional branches, sub-routine calls.
Input / Output	IN, OUT	Exchange data with peripherals (keyboard, screen, disk).

EXEMPLE **Reading a simple instruction**

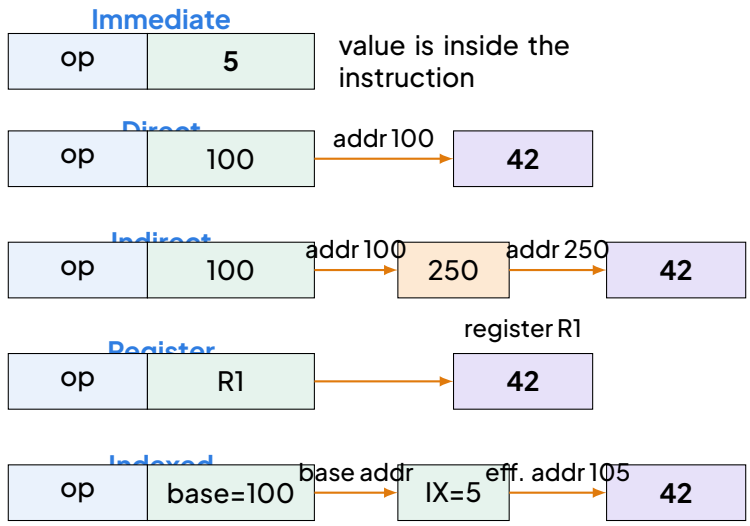
`LOAD R1, 100` means: copy into register R1 the contents of memory address 100. `ADD R1, R2` means: R1 ← R1 + R2. `JZ 40` means: if the last result was zero, jump to the instruction at address 40; otherwise continue with the next instruction.

2.3 Addressing Modes

When an instruction needs data, where does the operand actually live? The **addressing mode** answers this question. This is one of the most frequently examined notions in this chapter.

- **Immediate addressing**: the operand field is the value itself. `MOV R1, #5` puts the number 5 into R1. Fast, because no memory access is needed, but the value is fixed in the program.
- **Direct (absolute) addressing**: the operand field is the memory address of the value. `LOAD R1, 100` fetches the contents of cell 100.

- **Indirect addressing**: the operand field is the address of a cell that itself contains the address of the value. `LOAD R1, (100)` reads cell 100, finds (say) 250 there, and finally loads the contents of cell 250. This is how pointers work.
- **Register addressing**: the operand is in a register. `ADD R1, R2` uses the values already inside R1 and R2. This is the fastest mode after immediate.
- **Indexed addressing**: the effective address is a base address plus the contents of an index register: effective address = base + index. Perfect for walking through arrays: keep the base fixed and increment the index.



Where the operand really lives in each addressing mode (the value wanted is 42).

WARNING **Immediate vs direct — the classic GCE trap**

In immediate addressing the operand IS the value: `MOV R1, #5` puts 5 in R1. In direct addressing the operand is the ADDRESS of the value: `LOAD R1, 5` puts into R1 whatever is stored in memory cell 5 — which could be anything. Confusing the two is the most common error in paper 2 questions on addressing modes. Look for the # symbol (or the wording of the question) to decide.

2.4 The Fetch–Decode–Execute Cycle in Detail

How does the CPU actually run a program? It repeats forever a three-step rhythm called the **fetch–decode–execute cycle** (or instruction cycle). Four special registers orchestrate it: the **PC** (Program Counter, address of the next instruction), the **MAR** (Memory Address Register), the **MDR** (Memory Data Register) and the **IR** (Instruction Register).

METHOD **Tracing the instruction cycle**

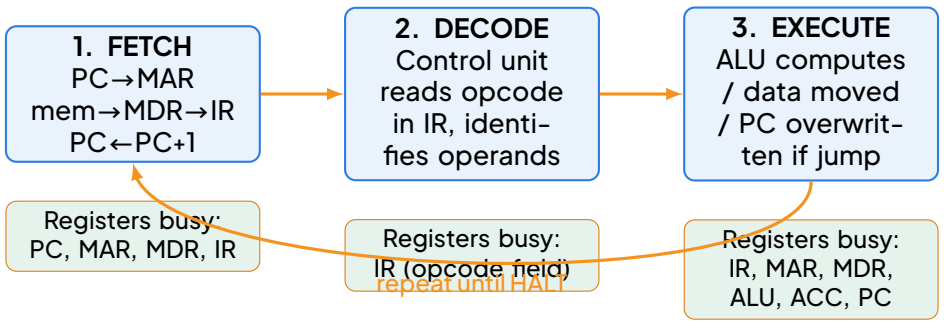
To trace one cycle, follow the PC:

1. Fetch. The PC is copied into the MAR (`PC → MAR`); the address travels on the address bus; memory returns the instruction into the MDR (`memory → MDR`); the MDR is copied into the IR (`MDR → IR`); the PC is incremented to point to the next instruction.
2. Decode. The control unit examines the opcode in the IR, identifies the operation, and works out where the operands are (addressing mode).
3. Execute. The operation is carried out: data may be fetched (a second memory access through MAR/MDR), the ALU may compute, a result is stored, or the PC is overwritten by a jump.

Then the cycle restarts with the new value of the PC.

WARNING **The PC is incremented during fetch**

The PC is incremented during the fetch stage, not after execute. So when a jump instruction is executed, the PC already holds the address of the next sequential instruction, and the jump simply overwrites it. When you trace a program containing `JMP` or `JZ`, write the PC value carefully at each stage — examiners award marks for the intermediate values.



The fetch–decode–execute cycle with the registers involved at each stage.

WORKED EXAMPLE 3 **Tracing a 3-instruction program with trace table**

Memory contains: at address 10: `LOAD ACC, 100`; at 11: `ADD ACC, 101`; at 12: `STORE ACC, 102`. Cell 100 holds 7, cell 101 holds 5. Initially PC = 10. Trace the cycles.

SOLUTION

We trace each cycle using a register trace table. The PC is incremented during the fetch phase.

Cycle	Phase	PC	MAR	MDR	IR	ACC
Start	–	10	–	–	–	?
1	Fetch	11	10	`LOAD ACC,100`	`LOAD ACC,100`	?
	Decode	11	10	`LOAD ACC,100`	`LOAD ACC,100`	?
	Execute	11	100	7	`LOAD ACC,100`	7
2	Fetch	12	11	`ADD ACC,101`	`ADD ACC,101`	7
	Decode	12	11	`ADD ACC,101`	`ADD ACC,101`	7
	Execute	12	101	5	`ADD ACC,101`	12
3	Fetch	13	12	`STORE ACC,102`	`STORE ACC,102`	12
	Decode	13	12	`STORE ACC,102`	`STORE ACC,102`	12
	Execute	13	102	12	`STORE ACC,102`	12

Final state: ACC = 12, memory cell 102 = 12, PC = 13 (pointing to the next instruction after the program).

2.5 Machine Code, Assembly Language and High-Level Languages

The CPU understands only binary: machine code. Writing `0001010101100100` by hand is painful and error-prone, so programmers invented **assembly language**, in which each binary instruction is written as a short mnemonic (`LOAD`, `ADD`, `JMP`…). The correspondence is exact:

PROPERTY **One-to-one mapping of assembly to machine code**

Each assembly language instruction corresponds to exactly one machine code instruction. By contrast, a single high-level language statement (for example `total = a + b * c;` in C) is generally translated into several machine instructions. (The mapping is examinable; you must be able to hand-assemble a short program.)

Level	Example	Characteristics
Machine code	`0001 0000 01100100`	Binary; executed directly; machine-dependent.
Assembly	`LOAD ACC, 100`	One mnemonic per machine instruction; needs an assembler.
High-level	`x = x + y;`	Close to English/maths; portable; needs a compiler or interpreter.

2.6 Translators: Assembler, Compiler, Interpreter

A **translator** converts source code into a form the machine can run. There are three kinds:

- **Assembler**: translates assembly language into machine code, instruction by instruction (one-to-one). Used for device drivers, embedded systems, and the labs of Lessons 91–92.
- **Compiler**: translates the whole high-level program into machine code in one go, producing an executable file. Execution is then very fast, but errors are reported only after the whole program is analysed. Examples: C, C++.
- **Interpreter**: translates and executes the program line by line, every time it runs. No executable is produced; execution is slower, but errors are located immediately and the same source runs on any machine that has the interpreter. Examples: Python, early BASIC.

DID YOU KNOW? **Why two strategies?**

A compiled C program controlling an electricity meter in Douala starts instantly and runs at full speed — ideal for embedded devices. A Python script used by a student in Bamenda to analyse experiment data is easier to write, test and correct line by line. Choosing between compiler and interpreter is a genuine engineering decision, not just a matter of taste.

2.7 Interrupts: Suspending and Resuming Execution

While the CPU is looping through fetch–decode–execute, the outside world does not wait: a key is pressed, a disk finishes a read, a timer ticks. An **interrupt** is a hardware signal that forces the CPU to pause its current program. The mechanism, in brief:

1. The CPU finishes the current instruction, then saves its context — above all the PC and the other registers — onto a special memory area called the **stack**.
2. It jumps to the interrupt service routine (ISR), a small program that handles the event (for example, reading the key that was pressed).
3. When the ISR ends, the saved context is restored: the PC recovers its old value and the interrupted program resumes exactly where it stopped, as if nothing had happened.

Interrupts make the computer responsive: without them the CPU would have to waste time constantly polling every device.

2.8 Worked Example: Hand-Assembly into Binary Machine Code

We finish with the complete journey from an idea to pure binary — a typical GCE-style exercise. Suppose our 16-bit machine uses this opcode table:

Mnemonic	Opcode (binary)	Meaning
LOAD	0001	ACC ← memory[addr]
ADD	0010	ACC ← ACC + memory[addr]
STORE	0011	memory[addr] ← ACC
HALT	1111	stop

Task: add the numbers stored in cells 100 and 101, store the result in cell 102, and stop. The program is loaded from address 10.

Step 1 — write the assembly:

```
10: LOAD 100
11: ADD 101
12: STORE 102
13: HALT
```

Step 2 — convert each address to 12-bit binary:

$$100 = 64 + 32 + 4 = 2^6 + 2^5 + 2^2 = 0000\ 0110\ 0100_2$$
$$101 = 0000\ 0110\ 0101_2$$
$$102 = 0000\ 0110\ 0110_2$$

Step 3 — concatenate opcode and operand:

Address	Assembly	Machine code (16 bits)
10	LOAD 100	0001 0000 0110 0100
11	ADD 101	0010 0000 0110 0101
12	STORE 102	0011 0000 0110 0110
13	HALT	1111 0000 0000 0000

(HALT has no operand, so its operand field is padded with zeros.) These four 16-bit words are exactly what would be stored in memory and executed, one fetch-decode-execute cycle at a time.

EXERCISE 2 **Hand-assembly practice**

Using the same opcode table, hand-assemble into binary the following program, loaded at address 20: `LOAD 200`, `ADD 201`, `ADD 202`, `STORE 203`, `HALT`. Then trace the five cycles giving the contents of PC, MAR, MDR, IR and ACC at each step, given that cells 200, 201 and 202 contain 9, 4 and 7 respectively.

SOLUTION **Exercise 1**

Addresses: 200 = $0000\,1100\,1000_2$, 201 = $0000\,1100\,1001_2$, 202 = $0000\,1100\,1010_2$, 203 = $0000\,1100\,1011_2$. Machine code: `0001 0000 1100 1000; 0010 0000 1100 1001; 0010 0000 1100 1010; 0011 0000 1100 1011; 1111 0000 0000 0000.`

Trace (PC increments during each fetch: 20 → 21 → 22 → 23 → 24 → 25):

Cycle	Phase	PC	MAR	MDR	IR	ACC
Start	–	20	–	–	–	?
1 (LOAD)	Fetch	21	20	`LOAD 200`	`LOAD 200`	?
	Execute	21	200	9	`LOAD 200`	9
2 (ADD)	Fetch	22	21	`ADD 201`	`ADD 201`	9
	Execute	22	201	4	`ADD 201`	13
3 (ADD)	Fetch	23	22	`ADD 202`	`ADD 202`	13
	Execute	23	202	7	`ADD 202`	20
4 (STORE)	Fetch	24	23	`STORE 203`	`STORE 203`	20
	Execute	24	203	20	`STORE 203`	20
5 (HALT)	Fetch	25	24	`HALT`	`HALT`	20
	Execute	25	–	–	`HALT`	20

After LOAD, ACC = 9; after first ADD, ACC = 13; after second ADD, ACC = 20; STORE writes 20 into cell 203; HALT stops the machine with PC = 25 (pointing to the next instruction after HALT).

KEY POINTS **Key points of Lesson 90**

- The ISA is the complete set of machine instructions, their formats and addressing modes; every instruction = opcode + operand(s).
- Four instruction families: data transfer, arithmetic/logic, control flow, I/O.
- Addressing modes: immediate (operand is the value), direct (operand is the address), indirect (address of an address), register, indexed (base + index).
- The cycle is fetch (PC→MAR, memory→MDR→IR, PC incremented) — decode — execute; the PC changes during fetch.
- Assembly maps one-to-one to machine code; high-level statements compile to many instructions. Assembler, compiler (whole program, executable) and interpreter (line by line) are the three translators.
- Interrupts save the PC and registers on the stack, run a service routine, then restore

and resume.

In the next section we leave theory for practice: you will build, in C, a working simulation of this very cycle — a small virtual CPU with its own memory, registers and instruction set.

3 SECTION 3: HANDS-ON C PROJECT — CPU INSTRUCTION CYCLE & MEMORY SIMULATION (LESSON 91)

In Section 2 we studied the Instruction Set Architecture of a processor: the opcodes, the registers, and the fetch–decode–execute cycle that the control unit performs endlessly. Theory, however, becomes truly convincing only when you build the machine yourself. In this hands-on project we shall write, in the C language, a complete simulator of a tiny CPU: a machine with 16 bytes of memory, three registers (PC, IR, ACC) and a four-instruction set (LOAD, ADD, STORE, HALT). By the end of the lesson you will have watched, line by line on your screen, exactly the same cycle that the processor of the computer in front of you executes billions of times per second.

3.1 Project specification

DEFINITION **The tiny CPU to simulate**

We build a program that simulates a processor with the following characteristics:

- Memory: 16 bytes, addressed from 0 to 15.
- Registers: PC (Program Counter), IR (Instruction Register), ACC (Accumulator).
- Instruction set: LOAD (load a memory byte into ACC), ADD (add a memory byte to ACC), STORE (write ACC into memory), HALT (stop the machine).

Why such a small machine? Because every concept of a real processor — the program counter, the instruction register, the accumulator, the fetch–decode–execute cycle — is already present. A real CPU simply has more memory, more registers and more instructions; the principle is identical. This is the same reasoning an engineer uses when testing a design on a small prototype before building the full system.

3.2 Representing memory and registers in C

The memory of our machine is a sequence of 16 bytes. In C, the natural model is an array of bytes; the registers are simple variables:

- `unsigned char mem[16];` // the 16 bytes of memory
- `unsigned char PC = 0;` // Program Counter: address of next instruction

- `unsigned char IR = 0;` // Instruction Register: current instruction
- `unsigned char ACC = 0;` // Accumulator: where results live
- `int halted = 0;` // 1 when HALT has been executed

WARNING **Use unsigned char for memory bytes**

Always declare memory bytes as `unsigned char`. A plain `char` in C is usually signed: any value above 127 is then interpreted as a negative number. If memory cell 5 holds the value 200, a signed `char` would report -56, silently corrupting every computation in your simulation. With `unsigned char`, the range is exactly 0 to 255, which is what a byte means in hardware.

WARNING **Address 0 is a valid address**

Array indices in C start at 0. Memory address 0 is a perfectly valid address: the first instruction of a program normally lives there. A very common beginner's mistake is to start loading the program at `mem[1]` while the PC starts at 0 — the simulator then executes whatever garbage byte happens to be at address 0. Do not skip address 0.

3.3 Encoding instructions as bytes

A real machine stores each instruction as binary patterns in memory. Our instructions each need two pieces of information: what to do (the operation) and where (the memory address of the data). Since our memory has only 16 addresses, an address fits in 4 bits (because $2^4 = 16$); and since we have only 4 instructions, the operation also fits in 4 bits (because $2^2 = 4 \le 2^4$). One byte is therefore exactly enough.

DEFINITION **Opcode encoding**

Opcode encoding consists in packing the operation and its operand into a single byte: the high nibble (the 4 most significant bits) holds the **opcode** identifying the operation, and the low nibble (the 4 least significant bits) holds the **operand address**. If o is the opcode and a the address, the encoded byte is

$$\text{byte} = (o \ll 4) \mid a = 16\,o + a.$$

We choose the following encoding (our own miniature ISA):

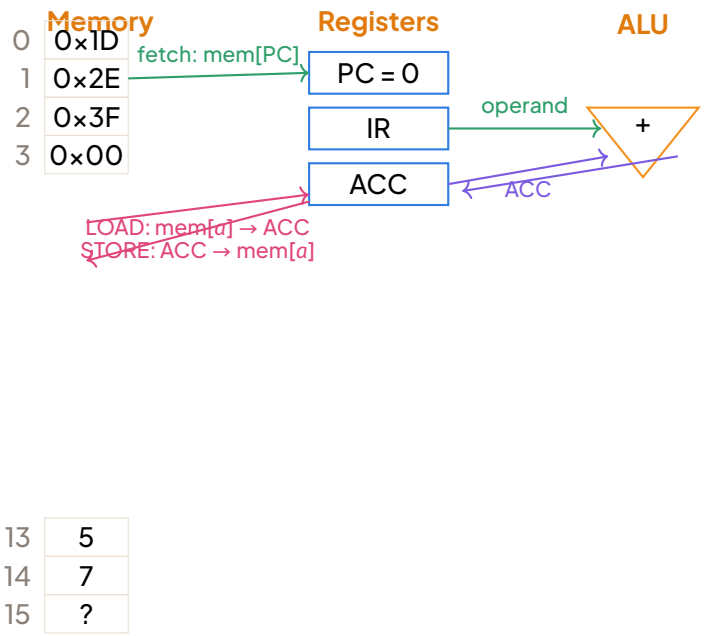
Mnemonic	Opcode (hex)	Opcode (binary)	Meaning of `opcode, address`
LOAD a	1	0001	ACC ← mem[a]
ADD a	2	0010	ACC ← ACC + mem[a]
STORE a	3	0011	mem[a] ← ACC
HALT	0	0000	stop the machine (operand ignored)

EXEMPLE **Encoding a small program by hand**

We want the machine to compute 5 + 7 and store the result at address 15. Place the data 5 and 7 at addresses 13 and 14. The program is:

- LOAD 13 → opcode 1, address 13: byte = 16 × 1 + 13 = 29 (hex `0x1D`)
- ADD 14 → opcode 2, address 14: byte = 16 × 2 + 14 = 46 (hex `0x2E`)
- STORE 15 → opcode 3, address 15: byte = 16 × 3 + 15 = 63 (hex `0x3F`)
- HALT → opcode 0: byte = 0 (hex `0x00`)

So `mem[0]=0x1D; mem[1]=0x2E; mem[2]=0x3F; mem[3]=0x00; mem[13]=5; mem[14]=7;` and the expected final result is mem[15] = 12.



The simulated machine: a 16-byte memory, the registers PC, IR and ACC, and the data movements during fetch, LOAD and STORE.

3.4 Implementing fetch, decode and execute

Fetch. The control unit reads the instruction pointed to by PC into the IR, then advances PC so that it points to the next instruction:

- `IR = mem[PC];` // fetch the instruction
- `PC = PC + 1;` // point to the next one

Note the order: PC is incremented during fetch, immediately after the instruction has been copied into IR. This detail will matter when we introduce jumps: a jump instruction will overwrite PC with a new address, and that new address must be the one used at the next fetch.

Decode. The instruction byte must now be split into its two nibbles. This is exactly the reverse of the encoding: shift right by 4 bits to recover the high nibble, and mask with `0x0F` (binary 00001111) to keep only the low nibble:

- `opcode = IR >> 4;` // high nibble: e.g. 0×1D » 4 = 0×1
- `operand = IR & 0x0F;` // low nibble: 0×1D & 0×0F = 0×D = 13

Recall why these work. The operator `>>` shifts every bit to the right, discarding the 4 lowest bits: `00011101` becomes `00000001`. The operator `&` keeps a bit only where the mask has a 1: `00011101 & 00001111 = 00001101`, which is 13. These two bit operations are the software image of the wiring that, in hardware, connects the high nibble of IR to the control unit and the low nibble to the address bus.

Execute. Depending on the opcode, the appropriate action is performed. A `switch` statement is the natural structure:

```
switch (opcode) {
  case 1: ACC = mem[operand]; break;   // LOAD
  case 2: ACC = ACC + mem[operand]; break;   // ADD
  case 3: mem[operand] = ACC; break;   // STORE
  case 0: halted = 1; break;   // HALT
  default: printf("Illegal opcode %d\n", opcode); halted = 1; }
}
```

The `default` case is important: if memory contains a byte whose high nibble is not 0, 1, 2 or 3 (for example an uninitialised cell), the simulator reports it and stops instead of silently doing nothing. Real processors behave similarly: an unknown opcode raises an illegal instruction exception.

PROPERTY **The switch mirrors the control unit**

A `switch` on the opcode mirrors exactly what the **control unit** does in hardware during decoding: it examines the opcode bits and routes the machine towards the correct circuit

(the ALU for ADD, the data bus for LOAD/STORE, the clock stop for HALT). In a real CPU this routing is done by logic gates; in our simulator it is done by the `switch`. The correspondence is one-to-one.

METHOD **Structure the simulator as a cycle loop**

The whole simulator is the fetch–decode–execute cycle of Section 2, written as a loop:

```
while (!halted) {
    fetch();    // IR = mem[PC]; PC++;
    decode();   // opcode = IR » 4; operand = IR & 0×0F;
    execute();  // the switch on the opcode
    print_state();   // show PC, IR, ACC, mem[15]
}
```

Each iteration of the loop is exactly one machine cycle.

3.5 Tracing execution: a debugging skill

The function `print_state()` is not a luxury: it is your microscope. After each cycle, print the cycle number, PC, IR (in hex), ACC and the memory cell that will hold the result:

```
printf("cycle %d:  PC=%2d IR=0x%02X ACC=%3d mem[15]=%3d\n",
       cycle, PC, IR, ACC, mem[15]);
```

Running the simulator on our test program produces the following trace:

Cycle	PC (after)	IR	ACC	Action performed	mem[15]
1	1	0×1D	5	LOAD 13: ACC ← mem[13] = 5	0
2	2	0×2E	12	ADD 14: ACC ← 5 + 7	0
3	3	0×3F	12	STORE 15: mem[15] ← 12	12
4	4	0×00	12	HALT: machine stops	12

The final value mem[15] = 12 = 5 + 7 confirms that the simulator is correct.

METHOD **Test incrementally**

Always test with the smallest possible program first:

1. Put only `HALT` (`mem[0] = 0x00`) in memory. The simulator must stop after exactly one cycle. If it does not, the loop or the HALT case is wrong.
2. Add a single `LOAD` and check that ACC receives the right value.
3. Then add `ADD`, then `STORE`, verifying the trace after each addition.

This incremental strategy localises every bug to the last instruction you added — a habit worth keeping for every program you will ever write.

WORKED EXAMPLE 4 **Worked exercise: hand-tracing a program**

The memory of our tiny CPU is loaded as follows: `mem[0]=0x1A; mem[1]=0x2B; mem[2]=0x3C; mem[3]=0x00; mem[10]=4; mem[11]=9;` with all other bytes 0. Give the complete trace (PC after fetch, IR, ACC, mem[12]) and the final value of `mem[12]`.

SOLUTION

Solution. Decode each byte: `0x1A` = LOAD 10; `0x2B` = ADD 11; `0x3C` = STORE 12; `0x00` = HALT.

Cycle	PC	IR	ACC	Action
1	1	0×1A	4	ACC ← mem[10] = 4
2	2	0×2B	13	ACC ← 4 + mem[11] = 4 + 9
3	3	0×3C	13	mem[12] ← 13
4	4	0×00	13	HALT

Final value: mem[12] = 13 = 4+9. Note how the operand nibble (A, B, C in hex) directly gives the addresses 10, 11, 12 — this is why hexadecimal is the natural language of machine code.

3.6 Extension (extra): conditional branching with JZ

DID YOU KNOW? **Going further: a zero flag and a JZ instruction**

Real programs need decisions. Add to your simulator:

- a zero flag `int ZF;` set to 1 whenever ACC becomes 0 after LOAD or ADD, and 0 otherwise;
- a new instruction JZ *a* (opcode 4): jump if zero — if `ZF == 1` then `PC = a`, otherwise do nothing.

The execute case is simply `case 4: if (ZF) PC = operand; break;`. Because PC was already incremented during fetch, writing `PC = operand` cleanly redirects the next fetch to address *a*. With JZ you can write loops: for example, a program that counts down from 5 to

0. You have just invented conditional branching, the mechanism behind every `if` and every `while` you have ever written in C.

PROPERTY **Byte arithmetic wraps around modulo 256**

Since ACC is an `unsigned char`, every result is taken modulo 256: 255 + 1 gives 0, and adding 255 is the same as subtracting 1, because $-1 \equiv 255 \pmod{256}$. This is exactly the two's-complement behaviour of a real 8-bit ALU, and it is what makes a "decrement" possible even though our ISA has no SUB instruction. (Statement to know; the proof is not examinable.)

WARNING **Common traps in this project**

- Forgetting `break;` at the end of a `case`: execution falls through into the next case and two instructions are executed in one cycle.
- Incrementing PC after execute instead of during fetch: JZ-type extensions then jump to the wrong address.
- Printing `IR` with `%d` instead of `%02X`: the trace becomes much harder to read against the encoding table.
- Using `int` for `mem` and then wondering why the "byte" metaphor breaks: keep memory as `unsigned char` to stay faithful to hardware.
- Loading data bytes into the same addresses as the program: the simulator would then execute your data as if it were code. Keep program (low addresses) and data (high addresses) clearly separated.

EXERCISE 3 **Your turn**

1. Encode by hand, in hexadecimal, the program: LOAD 12, ADD 13, ADD 14, STORE 15, HALT, with data `mem[12]=3`, `mem[13]=4`, `mem[14]=10`. Predict the final value of `mem[15]` before running the simulator.
2. Implement the full simulator in C with the `print_state()` trace, and verify your prediction.
3. (Extension) Add the zero flag and the JZ instruction, then write a program that adds `mem[14]` to ACC repeatedly while decrementing `mem[13]` until it reaches zero.

SOLUTION **Exercise 1**

1. Encodings: LOAD 12 = `0x1C`; ADD 13 = `0x2D`; ADD 14 = `0x2E`; STORE 15 = `0x3F`; HALT = `0x00`. The computation is 3 + 4 + 10 = 17, so the predicted final value is mem[15] = 17.
2. The trace must show ACC taking the values 3, 7, 17, then `mem[15]` becoming 17 at the STORE cycle, then HALT. Any other trace indicates a bug to locate with the incremental method.
3. Hint for the loop: LOAD 13; ADD the constant 255 (since $-1 \equiv 255 \pmod{256}$, this decrements ACC); STORE 13; JZ to the HALT address; otherwise jump back — note that a plain JUMP (opcode 5, `case 5: PC = operand; break;`) makes this easier. This is genuine assembly-level thinking.

KEY POINTS **Key points of Section 3**

- A CPU simulator models memory as `unsigned char mem[16]` and registers as variables; the whole machine is one `while(!halted)` loop of fetch, decode, execute, print.
- Instructions are packed into one byte: high nibble = opcode, low nibble = operand; decoding uses `IR >> 4` and `IR & 0x0F`.
- The `switch` on the opcode plays the role of the hardware control unit; PC is incremented during fetch, before execute.
- Byte arithmetic is modulo 256, exactly like a real 8-bit ALU; adding 255 decrements by 1.
- Tracing the state after every cycle is the essential debugging tool; test incrementally, starting from HALT alone.
- Address 0 is valid; always use `unsigned char` for bytes.

This simulator has given you a working machine built entirely in software. In Section 4 we move one step closer to the real hardware: writing actual assembly code and manipulating registers directly, to feel what the programmer's view of the processor truly is.

4 SECTION 4: HANDS-ON C PROJECT — ASSEMBLY CODE & REGISTER MANIPULATION LAB (LESSON 92)

In Section 3 we built a C simulator that fetched, decoded and executed machine instructions stored as bytes in a simulated memory. But where do those bytes come from? No human programmer enjoys writing raw binary. Programmers write assembly language — short, readable

mnemonics such as `ADD R1, R2` — and a program called an assembler translates each line into the byte encoding that the CPU (or our simulator) understands. In this final lab we close the loop: we learn to write and hand-trace small assembly routines, we master the C bitwise operators that mimic the ALU's logic operations, and we finish with a mini-assembler whose output feeds directly into the Section 3 simulator. This is also the most examinable skill of the chapter: GCE papers regularly ask you to trace a short assembly listing and state the final register contents.

4.1 A Simple Assembly Syntax and the Register Bank

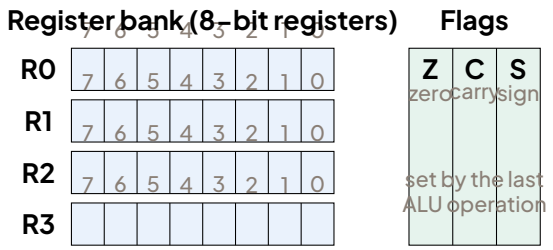
Recall from Lesson 90 that an **assembly instruction** consists of a mnemonic (the operation) followed by operands (registers or immediate values). Our hypothetical processor exposes a tiny **register bank** of four 8-bit general-purpose registers `R0`–`R3`, plus a flags register recording the state of the last ALU operation.

DEFINITION **Register bank**

A register bank is the small set of general-purpose registers directly accessible to assembly instructions. It is the fastest storage in the machine: the ALU reads its operands from registers and writes its result back to a register, without touching main memory.

DEFINITION **Flags register**

A flags register (or status register) is a special register whose individual bits are set or cleared automatically after each ALU operation. We use three flags: Z (Zero) is set when the result is 0; C (Carry) is set when an addition overflows 8 bits; S (Sign) is a copy of bit 7 of the result. Conditional jump instructions test these flags to make decisions.



The register bank R0–R3 (bit positions 7 down to 0) and the flags register holding the Zero, Carry and Sign flags.

Our assembly dialect uses the following instructions, each mapping to one byte encoding in the Section 3 simulator. We number the instructions from address 0, so that jump targets are simply line numbers.

Mnemonic	Example	Meaning
`MOV Rd, val`	`MOV R1, 5`	load immediate value into `Rd`
`MOV Rd, Rs`	`MOV R0, R1`	copy register `Rs` into `Rd`
`ADD Rd, Rs`	`ADD R1, R2`	`Rd ← Rd + Rs`, update flags
`SUB Rd, Rs`	`SUB R1, R2`	`Rd ← Rd - Rs`, update flags
`MUL Rd, Rs`	`MUL R1, R2`	`Rd ← Rd × Rs` (low 8 bits)
`CMP Ra, Rb`	`CMP R1, R2`	compute `Ra - Rb`, set flags only; registers unchanged
`JMP addr`	`JMP 0`	unconditional jump to address `addr`
`JZ addr`	`JZ 6`	jump if the Zero flag is set
`JS addr`	`JS 4`	jump if the Sign flag is set (last result negative)

4.2 Writing Short Assembly Routines by Hand

EXEMPLE **Sum of two numbers**

Add 12 and 30, leaving the result in `R0`:

- `0: MOV R0, 12` ; R0 = 12
- `1: MOV R1, 30` ; R1 = 30
- `2: ADD R0, R1` ; R0 = 42, flags updated

EXEMPLE **Maximum of two numbers**

Leave max(a, b) in `R0`, with a initially in `R0` and b in `R1`:

- `0: CMP R0, R1` ; flags from R0 − R1
- `1: JZ 5` ; if equal, R0 is already the maximum
- `2: JS 4` ; if R0 − R1 < 0, then b is larger
- `3: JMP 5` ; R0 > R1: nothing to do
- `4: MOV R0, R1` ; copy b into R0
- `5:` (end)

The key idea: `CMP` performs a subtraction purely to set the flags; the conditional jumps `JZ` and `JS` then decide which branch to take.

EXEMPLE **Countdown loop**

Count `R1` down from 5 to 0 (assume `R0 = 0`):

- `0: MOV R1, 5` ; counter
- `1: MOV R2, 1` ; step
- `2: SUB R1, R2` ; R1 = R1 - 1 (loop body)
- `3: CMP R1, R0` ; compare with 0
- `4: JZ 6` ; exit when zero (address 6 is just past the end)
- `5: JMP 2` ; repeat the body

4.3 Hand-Tracing Assembly: the Trace Table Technique

The single most reliable exam technique is the **trace table**: one column per register and per flag, one row per executed instruction.

METHOD **Tracing assembly in the exam**

1. Draw one column per register (`R0`–`R3`) and per flag (Z, C, S), plus an "instruction executed" column.
2. Write the initial state on the first row.
3. Execute one instruction per row, writing only the values that change.
4. For a jump, write on the next row the instruction found at the target address — never skip the condition test.

WARNING **Check the condition first**

When tracing loops, check the jump condition before assuming the body executes again. Off-by-one errors — looping once too many or once too few — are the most common exam mistake. Ask yourself: does the body run when the counter is already 0? Also remember that `CMP` never modifies a register: it only sets flags.

WORKED EXAMPLE 5 **Tracing the countdown loop**

Trace `0: MOV R1,3 ; 1: MOV R2,1 ; 2: SUB R1,R2 ; 3: CMP R1,R0 ; 4: JZ 6 ; 5:`

`JMP 2` with `R0 = 0`. State the final value of `R1`.

SOLUTION

Step	Instruction	R0	R1	R2	Z
0	(start)	0	?	?	0
1	`MOV R1,3`	0	3	?	0
2	`MOV R2,1`	0	3	1	0
3	`SUB R1,R2`	0	2	1	0
4	`CMP R1,R0`	0	2	1	0
5	`JZ 6` (not taken)	0	2	1	0
6	`JMP 2`	0	2	1	0
7	`SUB R1,R2`	0	1	1	0
8	`CMP R1,R0`	0	1	1	0
9	`JZ 6` (not taken)	0	1	1	0
10	`JMP 2`	0	1	1	0
11	`SUB R1,R2`	0	0	1	1
12	`CMP R1,R0`	0	0	1	1
13	`JZ 6` (taken)	0	0	1	1

At step 11 the subtraction gives 0, so `SUB` itself sets Z = 1; `CMP` at step 12 confirms it. The loop body executes exactly 3 times; the final value of `R1` is 0, with the Zero flag set.

4.4 Bitwise Operators in C: a Model of ALU Logic

The ALU manipulates registers bit by bit. C gives us the same machinery through its **bitwise operators**: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (shift left), `>>` (shift right).

PROPERTY **Shifts as arithmetic**

For an unsigned value, shifting left by k bits multiplies the value by 2^k (provided no significant bit is lost), and shifting right by k bits divides it by 2^k (integer division). Shifts are the cheapest arithmetic the ALU can perform — a single wiring pattern, no carry propagation. (Proof not examinable; verify on examples.)

EXEMPLE **Shifts on 8-bit values**

$13 = 00001101_2$. Then $13 \ll 2 = 00110100_2 = 52 = 13 \times 2^2$, and $13 \gg 2 = 00000011_2 = 3 = \lfloor 13/4 \rfloor$.

METHOD **Bit manipulation recipes**

For bit *i* of register `r` (bit 0 = rightmost):

- Test: `(r >> i) & 1` — brings bit *i* to position 0, then isolates it.
- Set: `r | (1 << i)` — OR with a mask containing a single 1.
- Clear: `r & ~(1 << i)` — AND with a mask containing a single 0.
- Toggle: `r ^ (1 << i)` — XOR flips exactly that bit.

WARNING **The NOT trap**

In C, `~` flips all bits of the integer, including the leading ones of a 32-bit `int`. When simulating 8-bit registers, always mask the result with `0xFF`: e.g. `r = (~r) & 0xFF;`. Forgetting the mask produces values above 255 that no real 8-bit register could hold.

4.5 Lab Task 1: Set, Clear, Toggle and Test a Bit

EXPERIMENT **Bit surgery on a simulated register**

Represent a register by an `unsigned int` and implement the four recipes:

- `unsigned int set_bit(unsigned int r, int i) { return r | (1u << i); }`
- `unsigned int clear_bit(unsigned int r, int i) { return r & ~(1u << i); }`
- `unsigned int toggle_bit(unsigned int r, int i) { return r ^ (1u << i); }`
- `int test_bit(unsigned int r, int i) { return (r >> i) & 1u; }`

Test with `r = 0xB2` (that is $10110010_2 = 178$): setting bit 0 gives 10110011_2 (`0xB3`); clearing bit 4 gives 10100010_2 (`0xA2`); toggling bit 7 gives 00110010_2 (`0x32`). Print results with `%02X` to read them in hexadecimal.

4.6 Lab Task 2: Simulating the Flags

After an 8-bit addition `res = a + b` (computed in a wider C `int`), the flags are obtained as follows:

- **Zero flag (Z):** set if the low 8 bits of the result are 0: `(res & 0xFF) == 0`.
- **Carry flag (C):** set if the true sum exceeds 255: `res > 0xFF`.
- **Sign flag (S):** set if bit 7 of the 8-bit result is 1: `(res & 0x80) != 0`.

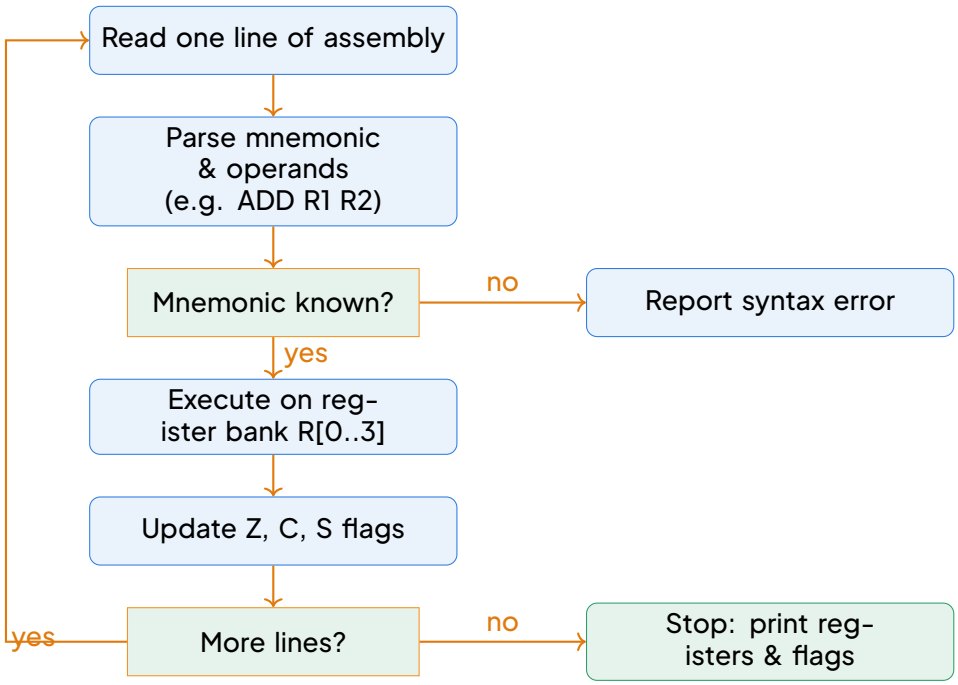
The value actually stored in the destination register is `res & 0xFF`.

EXEMPLE **Flags after** `200 + 100`

res = 300. Low 8 bits: 300 - 256 = 44 = `0x2C`, so the register holds 44 and Z = 0; 300 > 255 so C = 1; bit 7 of `0x2C` is 0 so S = 0. A real 8-bit ALU would store 44 and raise the carry — exactly how multi-byte (16-bit, 32-bit) addition is built from 8-bit blocks.

4.7 Lab Task 3: A Mini-Assembler

The final task assembles and executes our dialect. The register bank is an array `unsigned int R[4]`; and the flags three integers. The program loops: read a line, parse the mnemonic, execute, update registers and flags.



Flowchart of the mini-assembler: read line → parse mnemonic → execute → update registers and flags.

EXPERIMENT **Core of the mini-assembler**

Parsing can use `sscanf(line, "%s R%d R%d", op, &d, &s)` for register-register instructions and `"%s R%d %u"` for immediate loads. Execution is a chain of tests:

- `if (strcmp(op,"ADD")==0) R[d] = (R[d]+R[s]) & 0xFF;`
- `else if (strcmp(op,"SUB")==0) R[d] = (R[d]-R[s]) & 0xFF;`
- `else if (strcmp(op,"MOV")==0) R[d] = val & 0xFF;`
- …then recompute Z, C, S as in Lab Task 2.

Linking the two projects. Give each mnemonic a 4-bit opcode (e.g. `MOV` = 1, `ADD` = 2, `SUB` = 3) and each register a 2-bit code (`R0` = 0 to `R3` = 3). The mini-assembler then emits, for each register-register line, the byte encoding

$$\texttt{byte} = (\texttt{opcode} \ll 4) \mid (\texttt{dest} \ll 2) \mid \texttt{src}$$

— the opcode in the high nibble, the destination in bits 3–2 and the source in bits 1–0. For example `ADD R1, R2` becomes $(2 \ll 4) \mid (1 \ll 2) \mid 2 = \texttt{0x26}$. These are precisely the bytes that the fetch-decode-execute simulator of Section 3 stores in its memory array and decodes with the inverse operations `byte >> 4`, `(byte >> 2) & 3` and `byte & 3`. The toolchain is complete: assembly text → mini-assembler → machine bytes → CPU simulator.

DID YOU KNOW? **Assemblers in real life**

Every C program you compile passes through exactly this pipeline: the compiler translates C to assembly, the assembler translates assembly to machine code, and the linker combines the pieces into an executable file. Your two labs reproduce, in miniature, the tools that build every application on a Cameroonian student's phone.

4.8 GCE-Style Practice

METHOD **Golden rule for trace questions**

For GCE trace questions, draw one column per register and one row per instruction — never trace mentally. Mental tracing is how marks are lost on jumps and flags.

EXERCISE 4 **Timed trace (8 minutes)**

Given `R0 = 0` initially, trace the following and give the final values of `R1`, `R2` and the Z flag:

- `MOV R1, 4`

- `MOV R2, 3`
- `MUL R2, R2`
- `ADD R1, R2`
- `SUB R1, R2`
- `CMP R1, R0`

SOLUTION **Exercise 1**

Row by row: `R1=4`; `R2=3`; `R2=9` (3 × 3); `R1=13` (4 + 9); `R1=4` (13 − 9); `CMP` computes 4 − 0 = 4, so Z = 0. Final: `R1 = 4`, `R2 = 9`, Z = 0. Note that `CMP` does not modify `R1` — a classic trap.

EXERCISE 5 **Bit manipulation**

Write a C expression that (a) tests bit 3 of `reg`, (b) forces bits 0 and 7 to 1, (c) clears bit 5, keeping `reg` within 8 bits.

SOLUTION **Exercise 2**

(a) `(reg >> 3) & 1`; (b) `reg = reg | 0x81;` (bits 7 and 0: 10000001_2); (c) `reg = reg & (~(1 << 5)) & 0xFF;` — the final mask guarantees an 8-bit result.

EXERCISE 6 **Decoding a machine byte**

The Section 3 simulator reads the byte `0x26` using the encoding `(opcode << 4) | (dest << 2) | src` with opcodes `MOV` = 1, `ADD` = 2, `SUB` = 3. Which assembly instruction does it represent?

SOLUTION **Exercise 3**

`0x26` = 00100110_2. Opcode = `0x26 >> 4` = 2 (`ADD`); dest = (`0x26 >> 2`) & 3 = 1 (`R1`); src = `0x26` & 3 = 2 (`R2`). The instruction is `ADD R1, R2`.

KEY POINTS **Key points of Lesson 92**

- Assembly instructions act on a small register bank; flags (Z, C, S) record the outcome of the last ALU operation, and `CMP` sets flags without modifying any register.
- Trace tables — one row per instruction, one column per register/flag — are the safe exam method; always test loop conditions before re-entering the body.
- C bitwise operators model the ALU: `& | ^ ~ << >>`; left shift by k multiplies by 2^k, right shift divides by 2^k.
- Bit recipes: test `(r>>i)&1`, set `r|(1<<i)`, clear `r&~(1<<i)`, toggle `r^(1<<i)`; mask with `0xFF` for 8-bit registers.
- A mini-assembler parses mnemonics, executes them on a simulated register bank, and emits byte encodings `(opcode<<4)|(dest<<2)|src` for the Section 3 CPU simulator — completing the toolchain from assembly text to executing machine.

5 INTEGRATION ACTIVITY

6 COMPUTER ARCHITECTURE & HARDWARE ORGANIZATION — INTEGRATION PROJECT: CAMCPU-1

6.1 Aims of the activity

This integration activity closes the chapter by asking you to **build, program and defend** a complete simulated computer, **CamCPU-1**. By the end of the activity, each team will have:

- consolidated its understanding of the **Von Neumann architecture** (memory, control unit, ALU, registers, buses) by implementing each block in C;
- applied the **fetch–decode–execute cycle** to a real running program, with a visible trace of the **PC**, **IR** and **ACC** at every step;
- designed a small **instruction set architecture** (ISA) and written a **mini-assembler** translating mnemonics into machine code;
- practised low-level C skills: arrays as memory, bitwise manipulation, register simulation, and structured program design;
- developed **oral examination skills**: presenting a live demonstration and answering "what happens inside the CPU now?" questions, exactly in the style of the GCE practical interview.

KEY POINTS **Competence targeted**

Given a real-life problem, design and implement a simulated computer (memory, registers, instruction cycle, assembler), use it to solve the problem, and explain the internal behaviour of the machine at each clock step.

6.2 Situation

Mama Fotso's Electronics Shop, Mvog-Mbi Market, Yaoundé.

Mama Fotso sells phone accessories: chargers, earphones, memory cards and phone covers. Every evening she computes the day's total sales by hand in a notebook, and she often makes arithmetic errors. Her nephew, an Upper Sixth Computer Science student, tells her: "Auntie, even the simplest computer could do this for you. Let me show you how a CPU actually works by building a small simulated one."

Your team has been hired (symbolically!) to build **CamCPU-1**, a simulated 8-bit computer written in C, and to demonstrate it by running Mama Fotso's **sales-total program**.

6.3 Technical Specification of CamCPU-1

Component	Specification
Memory	256 bytes, addressed 0 to 255; each cell holds one 8-bit value
Registers	PC (program counter, 8-bit), IR (instruction register, 8-bit), ACC (accumulator, 8-bit)
Instruction format	1 byte: bits 7–4 = opcode, bits 3–0 = operand address (0–15)
Flags	Z flag: set to 1 whenever ACC becomes 0

6.4 Instruction Set Architecture (ISA)

Opcode	Mnemonic	Action
`0x1`	`LOAD n`	ACC ← MEM[*n*]
`0x2`	`STORE n`	MEM[*n*] ← ACC
`0x3`	`ADD n`	ACC ← ACC + MEM[*n*]
`0x4`	`SUB n`	ACC ← ACC - MEM[*n*]
`0x5`	`JZ n`	if Z = 1 then PC ← *n*
`0xF`	`HALT`	stop the machine

6.5 The Business Problem

Mama Fotso's sales for one day (in hundreds of FCFA, to fit in 8 bits) are stored in memory cells 8 to 11: `25`, `40`, `15`, `30`. The program must add these four values and **store the total at address 12**, then halt. (Expected total: 110, i.e. 11 000 FCFA.)

6.6 Deliverables

1. The C source code of the simulator and mini-assembler.
2. A live demonstration with a step-by-step trace.
3. A 10-minute oral defence before the class.

6.7 Tasks

1. **Architecture design.** Draw the block diagram of CamCPU-1 showing memory, PC, IR, ACC, the ALU and the buses. State the width (in bits) of each register and justify why the operand field of an instruction can only address memory cells 0 to 15.
2. **Memory and registers in C.** Declare the memory as an array of 256 unsigned 8-bit integers and the three registers as unsigned 8-bit variables. Explain why the `unsigned char` type is appropriate here.

3. **Encoding.** Write the function that packs an opcode and an operand into one byte using the formula byte = (opcode ≪ 4) | operand. Encode by hand `ADD 10` and `HALT`, then verify with your function.

4. **The instruction cycle.** Implement the fetch–decode–execute loop: fetch the byte at `MEM[PC]` into IR and increment PC; decode by extracting opcode (IR ≫ 4) and operand (IR & `0x0F`); execute using a `switch` on the opcode. After each cycle, print the trace line: PC, IR (in hexadecimal), ACC, Z.

5. **The mini-assembler.** Write a C function that reads a line typed by the user (e.g. `ADD 10`) and produces the corresponding byte code stored at the next free memory address. Accept at least the six mnemonics of the ISA.

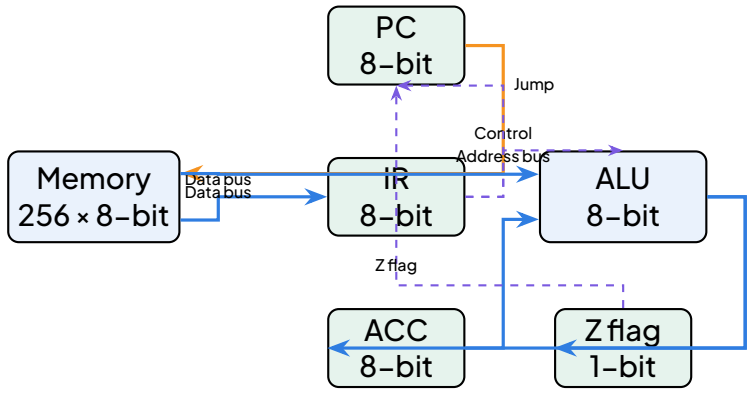
6. **The business program.** Using your assembler, enter the program: `LOAD 8`, `ADD 9`, `ADD 10`, `ADD 11`, `STORE 12`, `HALT`, starting at address 0, with the data at addresses 8–11. Run it and verify that MEM[12] = 110.

7. **Trace analysis.** Produce the full trace table of the run (one row per instruction) and identify, for each row, the fetch, decode and execute phases.

8. **Extension with JZ.** Modify the program so that it first subtracts a “daily expense” stored at address 13 (value 110): if the result is zero, jump to a routine that stores the message code `1` (“no profit”) at address 14, otherwise store code `2` (“profit”). Explain the role of the Z flag.

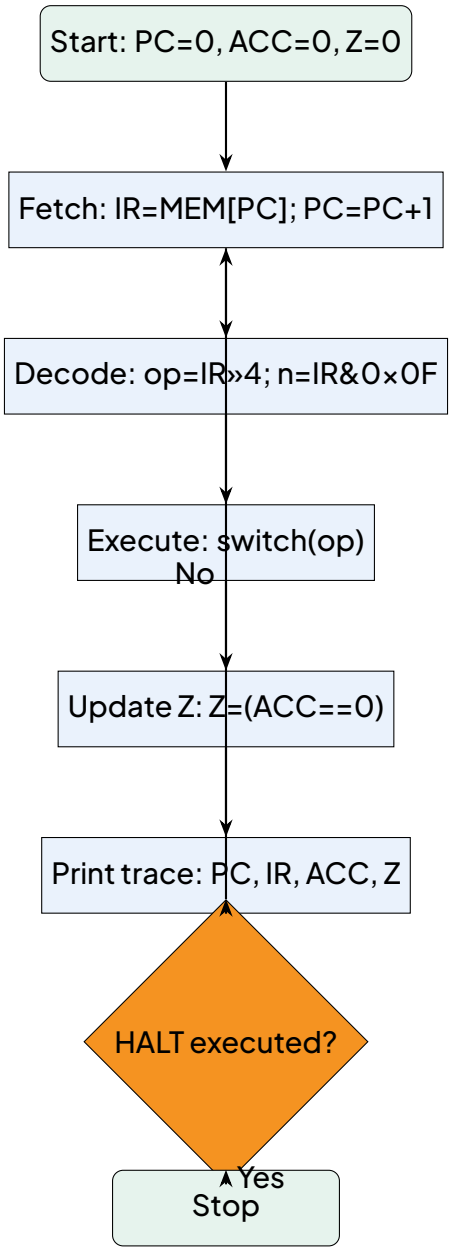
9. **Oral defence.** Prepare answers to: What is in the IR after the fetch of the third instruction? Which component performs the addition? What would happen if the total exceeded 255? Why is this a Von Neumann machine?

6.8 Block Diagram of CamCPU-1



CamCPU-1 Von Neumann architecture: single memory for program and data, shared address/data buses.

6.9 Fetch–Decode–Execute Cycle Flowchart



Fetch–Decode–Execute cycle with trace output.

6.10 Model Answers and Implementation

Task 1 – Architecture Design. CamCPU-1 follows the **Von Neumann model**: a single memory holds both program and data; the control unit fetches instructions via the address bus (driven by the PC) and receives them via the data bus into the IR; the ALU operates on the ACC and a memory operand. PC, IR and ACC are 8-bit registers. The operand field has 4 bits, and $2^4 = 16$, so only addresses 0 to 15 can be named directly inside an instruction. (Memory cells 16–255 are accessible only indirectly, e.g. by loading their address into a register and using self-modifying

code or an extended ISA — not required here.)

Task 2 – Memory and Registers in C. The declarations are:

```
unsigned char MEM[256];
unsigned char PC = 0, IR, ACC = 0;
unsigned char Z = 0;
```

`unsigned char` stores exactly 8 bits, values 0 to 255, matching the specification; overflow wraps around naturally (modulo 256), like real hardware. Using `signed char` would introduce negative values and undefined behaviour on overflow, which does not reflect actual CPU registers.

Task 3 – Encoding. The encoding function:

```
unsigned char encode(int op, int n) {
  return (op << 4) | (n & 0x0F); }
}
```

`ADD 10`: opcode 3, operand 10, byte = (3 ≪ 4) | 10 = `0x3A`. `HALT`: opcode 15, operand 0, byte = `0xF0`.

Task 4 – The Instruction Cycle. The core loop:

```
int running = 1;
while (running) {
  IR = MEM[PC]; PC = PC + 1;                                  // fetch
  int op = IR >> 4; int n = IR & 0x0F;                        // decode
  switch (op) {                                               // execute
    case 0x1: ACC = MEM[n]; break;
    case 0x2: MEM[n] = ACC; break;
    case 0x3: ACC = ACC + MEM[n]; break;
    case 0x4: ACC = ACC - MEM[n]; break;
    case 0x5: if (Z) PC = n; break;
    case 0xF: running = 0; break;
  }
  Z = (ACC == 0);
  printf("PC=%3d IR=%02X ACC=%3d Z=%d
n", PC, IR, ACC, Z);
}
```

Important: The Z flag is updated after the execute phase, so that `JZ` tests the flag resulting from the previous ALU operation — exactly as in real processors.

Task 5 – The Mini-Assembler. A simple line-oriented assembler:

```
void assemble_line(char *line, int *addr) {
  char mnemonic[10]; int operand = 0;
  if (sscanf(line, "%s %d", mnemonic, &operand) < 1) return;
  int op = 0;
  if (strcmp(mnemonic, "LOAD") == 0) op = 0x1;
  else if (strcmp(mnemonic, "STORE") == 0) op = 0x2;
  else if (strcmp(mnemonic, "ADD") == 0) op = 0x3;
  else if (strcmp(mnemonic, "SUB") == 0) op = 0x4;
  else if (strcmp(mnemonic, "JZ") == 0) op = 0x5;
  else if (strcmp(mnemonic, "HALT") == 0) op = 0xF;
  else { printf("Unknown mnemonic
n"); return; }
  MEM[*addr] = encode(op, operand);
  (*addr)++;
}
```

Task 6 – The Business Program. Memory image before the run:

Address	Content (hex)	Meaning
0	0x18	LOAD 8
1	0x39	ADD 9
2	0x3A	ADD 10
3	0x3B	ADD 11
4	0x2C	STORE 12
5	0xF0	HALT
8	25	sales data
9	40	sales data
10	15	sales data
11	30	sales data

After the run, `MEM[12] = 25+40+15+30 = 110`. Correct.

Task 7 – Trace Analysis. Full trace of the run:

Step	PC after fetch	IR (hex)	ACC	Execute action
1	1	0x18	25	ACC ← MEM[8] (25)
2	2	0x39	65	ACC ← 25 + MEM[9] (40)
3	3	0x3A	80	ACC ← 65 + MEM[10] (15)
4	4	0x3B	110	ACC ← 80 + MEM[11] (30)
5	5	0x2C	110	MEM[12] ← 110
6	6	0xF0	110	halt (running=0)

At each step the fetch phase copies `MEM[PC]` into the IR and increments the PC; the decode phase splits the IR into opcode and operand; the execute phase performs the action through the ALU or a memory transfer.

Task 8 – Extension with JZ. New program (addresses 0–10):

Addr	Code	Instruction
0	0x18	LOAD 8
1	0x39	ADD 9
2	0x3A	ADD 10
3	0x3B	ADD 11
4	0x4D	SUB 13 (expense = 110 at addr 13)
5	0x57	JZ 7 (jump to “no profit” routine)
6	0x1E	LOAD 14 (load profit code 2 from addr 14)
7	0x2F	STORE 15 (store result code at addr 15)
8	0xF0	HALT
9	0x1E	LOAD 14 (no profit: load code 1 from addr 14)
10	0x2F	STORE 15
11	0xF0	HALT

Data: addresses 8–11 as before; address 13 = 110 (expense); address 14 = 1 (no profit code) **or** 2 (profit code) — we preload both by placing 1 at address 14 and 2 at address 14? Wait, we need two different codes. Better: store code 1 at address 14, code 2 at address 15? Let’s redesign cleanly:

Clean extended program:

- Address 13: expense = 110
- Address 14: profit code = 2
- Address 15: no-profit code = 1
- Address 16: result destination

Program:

Addr	Code	Instruction
0	0x18	LOAD 8
1	0x39	ADD 9
2	0x3A	ADD 10
3	0x3B	ADD 11
4	0x4D	SUB 13 (ACC = 110−110 = 0, Z=1)
5	0x59	JZ 9 (jump to no-profit at addr 9)
6	0x1E	LOAD 14 (profit: load code 2)
7	0x20	STORE 16
8	0xF0	HALT
9	0x1F	LOAD 15 (no-profit: load code 1)
10	0x20	STORE 16
11	0xF0	HALT

After `SUB 13`, ACC = 0, so Z flag becomes 1; `JZ 9` therefore transfers control to address 9, where the "no profit" branch loads code 1 and stores it at address 16. If the expense were less than 110, ACC ≠ 0, Z=0, JZ does not jump, and the profit branch (addresses 6–8) executes, storing code 2. The Z flag is the hardware mechanism that lets the CPU make decisions: it records the result of the last ALU operation and the conditional jump tests it.

Task 9 – Oral Defence Answers.

1. After the fetch of the third instruction (address 2), IR contains `0x3A` (ADD 10).
2. The addition is performed by the **ALU**, with one operand from the ACC and the other fetched from memory via the data bus.
3. If the total exceeded 255, the 8-bit ACC would overflow and wrap around modulo 256 (e.g. 260 → 4). This is why Mama Fotso's amounts were scaled to hundreds of FCFA.
4. The machine is a Von Neumann machine because program and data share the same memory and are fetched over the same bus.

EXEMPLE **Complete CamCPU-1 Simulator Source Code (camcpu1.c)**

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MEMSIZE 256
unsigned char MEM[MEMSIZE];
unsigned char PC=0, IR=0, ACC=0, Z=0;
unsigned char encode(int op, int n) { return (op << 4) | (n & 0x0F); }
```

```
void fetch_decode_execute() {
  int running = 1;
  while (running) {
    IR = MEM[PC]; PC++;
    int op = IR >> 4; int n = IR & 0x0F;
    switch (op) {
      case 0x1:  ACC = MEM[n]; break;
      case 0x2:  MEM[n] = ACC; break;
      case 0x3:  ACC = ACC + MEM[n]; break;
      case 0x4:  ACC = ACC - MEM[n]; break;
      case 0x5:  if (Z) PC = n; break;
      case 0xF: running = 0; break;
    }
    Z = (ACC == 0);
    printf("PC=%3d IR=%02X ACC=%3d Z=%d\n", PC, IR, ACC, Z);
  }
}
void assemble_line(char *line, int *addr) {
  char mn[10]; int n=0;
  if (sscanf(line, "%s %d", mn, &n) < 1) return;
  for(int i=0; mn[i]; i++) mn[i] = toupper(mn[i]);
  int op = -1;
  if (strcmp(mn,"LOAD")==0) op=0x1;
  else if (strcmp(mn,"STORE")==0) op=0x2;
  else if (strcmp(mn,"ADD")==0) op=0x3;
  else if (strcmp(mn,"SUB")==0) op=0x4;
  else if (strcmp(mn,"JZ")==0) op=0x5;
  else if (strcmp(mn,"HALT")==0) op=0xF;
```

```
  if (op==-1) { printf("Unknown: %s\n", mn); return; }
  MEM[*addr] = encode(op, n);
  (*addr)++;
}
int main() {
  int addr = 0; char line[80];
  printf("CamCPU-1 Mini-Assembler. Enter lines, empty line to run.\n");
  while (fgets(line, sizeof(line), stdin) && line[0]!='\n') {
    assemble_line(line, &addr);
  }
  // Preload data for business problem
  MEM[8]=25; MEM[9]=40; MEM[10]=15; MEM[11]=30;
  // For extended version: MEM[13]=110; MEM[14]=2; MEM[15]=1;
  printf("\n--- Execution Trace ---\n");
  fetch_decode_execute();
  printf("\nMEM[12] = %d (total sales in hundreds of FCFA)\n", MEM[12]);
  return 0;
}
```

WARNING **Common mistakes to avoid**

- Forgetting to increment the PC during fetch (infinite loop on first instruction).
- Confusing the opcode nibble (bits 7–4) with the operand nibble (bits 3–0).
- Using `char` instead of `unsigned char` (negative values appear on overflow!).
- Updating the Z flag before testing it in `JZ` (the flag must reflect the previous ALU result).
- Placing data at addresses that the program later overwrites (e.g. using address 0 for data).
- Forgetting that the operand field is only 4 bits: `LOAD 20` will be truncated to `LOAD 4`.

EXERCISE 7 **Going further**

Add an instruction `INC n` (opcode `0x6`: MEM[*n*] ← MEM[*n*]+1) and use it to build a loop that counts how many of the four sales exceed 20 (hundred FCFA), storing the count at address 15. Present the trace to the class. Hint: You will need a counter in a memory cell, a loop using `JZ` after a `SUB` with a constant 20, and `INC` to increment the counter. Since CamCPU-1 has no immediate addressing, store the constant 20 in a free memory cell.

DID YOU KNOW? **Historical note**

The Von Neumann architecture (1945) introduced the concept of stored-program computers where instructions and data share the same memory. CamCPU-1 is a minimalist homage to the MANIAC I and EDVAC — the first machines to implement this idea. In Cameroon, the first mainframes (IBM 360) arrived in the 1970s for census and university research; today, every smartphone contains billions of transistors implementing the same fetch-decode-execute cycle you just simulated.

7 METHODS AND WORKED EXAMPLES

8 COMPUTER ARCHITECTURE & HARDWARE ORGANIZATION

8.1 Introduction and Syllabus Overview

This chapter covers the four lessons of the official Upper Sixth Computer Science syllabus (Third Term):

- **Lesson 89:** Computer Architecture & Hardware Organization — Von Neumann model, CPU components, system buses, memory hierarchy, fetch–decode–execute cycle.
- **Lesson 90:** Instruction Set Architecture & Low-Level Processing — Instruction formats, addressing modes, instruction sets (RISC/CISC), assembly language fundamentals.
- **Lesson 91:** Hands-on C Project: CPU Instruction Cycle & Memory Simulation — Modelling the CPU and RAM in C, tracing the instruction cycle.
- **Lesson 92:** Hands-on C Project: Assembly Code & Register Manipulation Lab — Writing and tracing assembly programs, register usage, control structures.

Each section below follows the pedagogical sequence: observation → explanation → worked examples → methods → exercises with corrections. GCE examination vocabulary and Cameroonian context are used throughout.

9 SECTION 1: COMPUTER ARCHITECTURE & HARDWARE ORGANIZATION (LESSON 89)

9.1 The Von Neumann Architecture — Observation and Model

EXPERIMENT **Opening a System Unit**

A student in Yaoundé opens the system unit of a desktop computer. She sees:

- A large rectangular board (the **motherboard**) with a socketed microprocessor chip under a heatsink/fan.
- Long thin modules plugged into slots (the **RAM** modules).
- A metal box with spinning platters (the **hard disk drive** or SSD).
- Flat ribbon cables or thin serial cables connecting the disk to the motherboard.

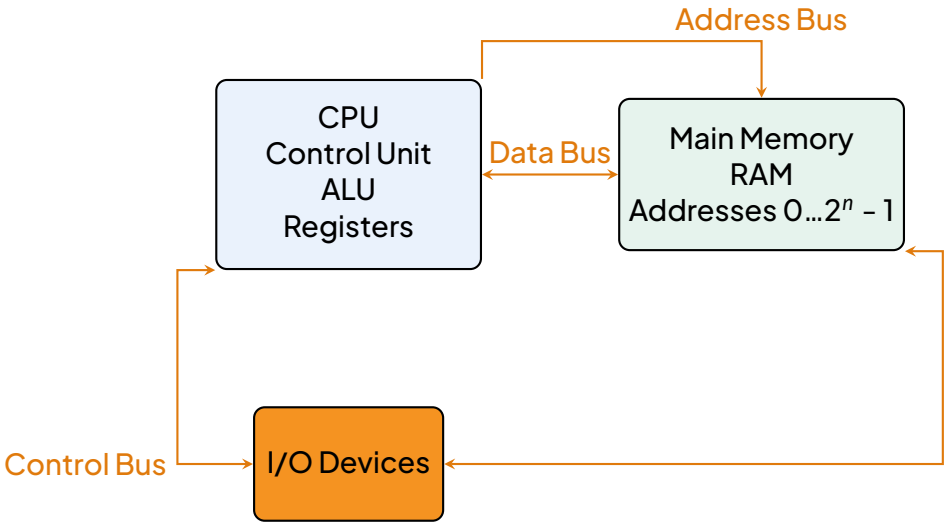
- A power supply unit with coloured wires.

Question: How do these physical parts correspond to the logical model taught in class?

DEFINITION **Von Neumann Architecture**

The **Von Neumann architecture** (1945) defines a stored-program computer with four logical subsystems:

1. Memory: a single addressable store holding both instructions and data (stored-program concept).
2. Central Processing Unit (CPU): fetches, decodes and executes instructions. It comprises the **Control Unit (CU)**, the **Arithmetic and Logic Unit (ALU)** and a set of **registers**.
3. Input/Output (I/O) devices: allow communication with the outside world (keyboard, screen, disk, network).
4. System buses: sets of parallel wires carrying addresses, data and control signals between the CPU, memory and I/O.



Von Neumann model: CPU, Memory, I/O connected by three buses.

9.2 CPU Components — Roles and Registers

PROPERTY **The Three Parts of the CPU**

The CPU consists of:

1. Control Unit (CU): generates timing and control signals; directs the fetch–decode–execute cycle; coordinates data movement between registers, ALU, memory and I/O.
2. Arithmetic and Logic Unit (ALU): performs arithmetic operations (add, subtract, multiply, divide) and logic operations (AND, OR, NOT, XOR, comparisons, shifts).
3. Registers: very fast, small-capacity storage locations inside the CPU, each with a dedicated role.

KEY POINTS **Key CPU Registers and Their Roles**

Register	Full Name	Role in the Instruction Cycle
PC	Program Counter	Holds the address of the next instruction to be fetched. Incremented after each fetch (or overwritten by a jump).
MAR	Memory Address Register	Holds the address of the memory location currently being accessed; drives the address bus.
MDR	Memory Data Register	Holds the data being read from or written to memory; connected to the data bus.
IR	Instruction Register	Holds the current instruction (opcode + operand) while it is being decoded and executed.
ACC	Accumulator	General-purpose register that holds one ALU operand and receives the result (implicit in many simple ISAs).
CIR	Current Instruction Register	Synonym for IR in some textbooks.
SR / PSR	Status Register / Processor Status Register	Holds **flags** (Zero, Carry, Sign, Overflow) set by the last ALU operation; used by conditional branches.

METHOD **Recognising and Explaining the Roles of CPU Components**

When a GCE question asks you to describe the architecture or the role of a component:

1. Draw the big picture first. Recall the Von Neumann model: one memory (instructions + data), one CPU (CU + ALU + registers), I/O, all linked by buses.
2. Classify the component. Does it control (CU), compute (ALU), store temporarily (register, cache, RAM), or transfer (bus)?
3. Name the buses precisely. Address bus (unidirectional, CPU → memory), data bus (bidirectional), control bus (READ, WRITE, INTERRUPT, etc.).
4. Give the role in one precise sentence mentioning what the component exchanges and with whom.
5. Illustrate with a concrete example (e.g. "the MAR holds the address 0×2F of the instruction being fetched").

Reflexes: MAR ↔ address bus; MDR ↔ data bus; PC holds the address of the next instruction; IR holds the current instruction being decoded.

WORKED EXAMPLE 6 **Identifying Components and Their Roles**

A student in Yaoundé opens the system unit of a desktop computer and identifies: a microprocessor chip, RAM modules, a hard disk, and flat cables connecting them.
(a) State the three main parts of the CPU.
(b) For each of the following registers, state its role: PC, IR, MAR, MDR, ACC.
(c) State the function of each of the three system buses.

SOLUTION

(a) Parts of the CPU. The Central Processing Unit consists of:

- the **Control Unit** (CU), which fetches, decodes and coordinates the execution of instructions;
- the **Arithmetic and Logic Unit** (ALU), which performs arithmetic (addition, subtraction) and logic (AND, OR, comparison) operations;
- a set of **registers**, very fast memory cells inside the CPU used to hold data and addresses being processed.

(b) Roles of the registers.

- PC (Program Counter): holds the address of the next instruction to be fetched from main memory. It is incremented after each fetch (or modified by a jump).
- IR (Instruction Register): holds the instruction currently being decoded and executed.

- MAR (Memory Address Register): holds the address of the memory location about to be read from or written to; it feeds the address bus.
- MDR (Memory Data Register): holds the data just read from memory or about to be written to memory; it is connected to the data bus.
- ACC (Accumulator): holds one operand of an ALU operation and receives the result (e.g. in `ADD 5`, 5 is added to the ACC).

(c) The three buses.

- Address bus: carries the memory address from the CPU (MAR) to memory; unidirectional. Its width determines the maximum addressable memory (an n-bit address bus addresses 2^n locations).
- Data bus: carries the actual data between CPU, memory and I/O; bidirectional.
- Control bus: carries timing and command signals (e.g. memory read, memory write, interrupt request) that coordinate all transfers.

9.3 System Buses — Width, Direction and Performance

PROPERTY **Bus Characteristics**

- Address bus width n: determines the number of addressable locations = 2^n. If memory is byte-addressable, maximum RAM = 2^n bytes.
- Data bus width w: number of bits transferred in one memory cycle. A wider data bus moves more bits per cycle → higher bandwidth.
- Control bus: carries signals such as `MEM_READ`, `MEM_WRITE`, `IO_READ`, `IO_WRITE`, `INTERRUPT`, `CLOCK`.

METHOD **Computing Memory Capacity and Bus Characteristics**

1. Key formula: an address bus of n bits can address 2^n memory locations.
2. Memory size = (number of locations) × (size of one location). If memory is byte-addressable, each location is 1 byte.
3. Reverse problem: to address M bytes, you need $n = \log_2 M$ address lines; round up to the next integer if M is not a power of two.
4. Convert units carefully: 1 KB = 2^{10} B, 1 MB = 2^{20} B, 1 GB = 2^{30} B.

5. Data bus width gives the number of bits transferred per cycle, hence influences speed/bandwidth, not capacity.

Reflex: always write the power of two explicitly before converting to KB/MB.

WORKED EXAMPLE 7 **Address Bus and Memory Capacity**

A microprocessor has a 16-bit address bus and an 8-bit data bus. Its memory is byte-addressable.
(a) How many distinct memory locations can it address?
(b) What is the maximum memory capacity, in KB?
(c) How many address lines are needed to address 4 MB of memory?
(d) Explain why doubling the data bus width to 16 bits does not increase the addressable memory.

SOLUTION

(a) Number of locations = 2^{16} = 65 536 locations.
(b) Each location holds 1 byte, so capacity = 65 536 bytes = $\frac{65\,536}{1024}$ = 64 KB.
(c) 4 MB = $4 \times 2^{20} = 2^{22}$ bytes. Hence $n = \log_2 2^{22} = 22$ address lines are required.
(d) The addressable space depends only on the address bus width (2^n locations). The data bus width determines how many bits are transferred in one memory operation: a 16-bit data bus moves 2 bytes per transfer instead of 1, which speeds up data transfer but leaves the number of addressable locations unchanged at 2^{16}.

WARNING **Common trap**

Do not confuse address bus width (determines capacity) with data bus width (determines transfer size per cycle). GCE questions often test this distinction.

9.4 The Fetch–Decode–Execute Cycle — Detailed Trace

DEFINITION **Instruction Cycle**

The **fetch–decode–execute cycle** is the fundamental operation of a Von Neumann CPU. For each instruction:

1. Fetch: PC → MAR; Memory[MAR] → MDR; MDR → IR; PC ← PC +1 (or PC ← PC + instruction length in bytes).
2. Decode: CU splits IR into **opcode** (operation code) and **operand** (address, immediate

value, or register specifier).

3. Execute: CU directs the ALU, registers and memory to perform the operation specified by the opcode.

After execute, the cycle repeats unless a HLT instruction stops the CPU.

METHOD **Building a Trace Table of the Instruction Cycle**

To answer "describe what happens when the instruction at address *A* is executed":

1. Write the initial state: PC, and the relevant memory contents.
2. Fetch phase: MAR ← PC; MDR ← Memory[MAR]; IR ← MDR; PC ← PC +1.
3. Decode phase: the CU splits the IR content into opcode and operand.
4. Execute phase: depends on the opcode — typical actions:
   - LOAD: MAR ← operand; MDR ← Memory[MAR]; ACC ← MDR.
   - STORE: MAR ← operand; MDR ← ACC; Memory[MAR] ← MDR.
   - ADD: MAR ← operand; MDR ← Memory[MAR]; ACC ← ACC + MDR.
   - JUMP: PC ← operand.
5. Present a trace table with one row per step and one column per register. Never skip the PC increment.

Reflex: the PC is incremented during fetch, so a JUMP overwrites an already-incremented PC.

WORKED EXAMPLE 8 **Tracing a Two-Instruction Program**

Main memory contains:

Address	Content	Meaning
10	LOAD 20	ACC ← Memory[20]
11	ADD 21	ACC ← ACC + Memory[21]
20	7	data
21	5	data

Initially PC = 10. Trace the fetch–decode–execute cycle for the two instructions, giving the contents of PC, MAR, MDR, IR and ACC after each step.

SOLUTION

Instruction 1: LOAD 20.

- Fetch: MAR ← PC = 10; MDR ← Memory[10] = "LOAD 20"; IR ← MDR; PC ← 11.
- Decode: the CU recognises opcode LOAD, operand = 20.
- Execute: MAR ← 20; MDR ← Memory[20] = 7; ACC ← 7.

Instruction 2: ADD 21.

- Fetch: MAR ← 11; MDR ← Memory[11] = "ADD 21"; IR ← MDR; PC ← 12.
- Decode: opcode ADD, operand = 21.
- Execute: MAR ← 21; MDR ← Memory[21] = 5; ALU computes 7 + 5; ACC ← 12.

Trace table:

Step	PC	MAR	MDR	IR	ACC
Initial state	10	–	–	–	–
Fetch LOAD 20	11	10	LOAD 20	LOAD 20	–
Execute LOAD 20	11	20	7	LOAD 20	7
Fetch ADD 21	12	11	ADD 21	ADD 21	7
Execute ADD 21	12	21	5	ADD 21	12

Final result: ACC = 12, PC = 12 (ready to fetch the next instruction).

9.5 Memory Hierarchy and Cache

DEFINITION **Memory Hierarchy**

Modern computers use a **memory hierarchy** to bridge the speed gap between the CPU (nanoseconds) and main memory (tens of nanoseconds) / disk (milliseconds):

Level	Typical Size	Access Time	Technology
Registers	< 1 KB	0.3–0.5 ns	Flip-flops (CPU)
L1 Cache	32–64 KB/core	0.5–1 ns	SRAM (on-chip)
L2 Cache	256 KB–2 MB/core	3–10 ns	SRAM (on-chip)
L3 Cache	8–64 MB (shared)	10–20 ns	SRAM (on-chip)
Main Memory (RAM)	4–128 GB	50–100 ns	DRAM (DIMM)
Secondary Storage	256 GB–8 TB	0.1–10 ms	SSD / HDD

PROPERTY **Cache Principle**

Cache is a small, fast memory (SRAM) placed between the CPU and main memory (DRAM). It stores copies of recently accessed memory locations.

- Cache hit: data found in cache → CPU proceeds at cache speed.
- Cache miss: data not in cache → fetch from RAM (slow), then copy into cache.
- Hit rate h: fraction of accesses that hit. Average access time = $h \times t_{\text{cache}} + (1 - h) \times t_{\text{RAM}}$.
- Locality of reference: temporal (recently used data likely used again) and spatial (data near recently used data likely used soon).

DID YOU KNOW? **Cameroon Context**

In many Cameroonian schools, laboratory computers have 4–8 GB RAM and modest L2/L3 caches (2–8 MB). When students run heavy IDEs (e.g. IntelliJ, PyCharm) alongside a browser, the small cache causes frequent misses, making the system feel slow even if the CPU clock is high. Upgrading RAM helps capacity; a CPU with larger cache helps latency.

9.6 Performance Factors — Clock, Cores, CISC vs RISC

PROPERTY **CPU Performance Equation**

$$\text{Execution Time} = \frac{\text{Instruction Count} \times \text{Cycles Per Instruction (CPI)}}{\text{Clock Frequency (Hz)}}$$

Factors influencing performance:

- Clock frequency: higher → shorter cycle time, but more heat/power.
- CPI: reduced by pipelining, superscalar execution, better ISA design.
- Instruction count: reduced by richer instruction sets (CISC) or better compilers (RISC).
- Number of cores: parallel execution of independent threads.
- Cache size/hierarchy: reduces effective memory latency, lowering CPI.
- Data bus width: wider bus → more bits per transfer → higher memory bandwidth.

KEY POINTS **RISC vs CISC — GCE Comparison Points**

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction set size	Small (30–100)	Large (100–300+)
Instruction length	Fixed (e.g. 32 bits)	Variable (1–15 bytes)
Addressing modes	Few (register, immediate, PC-relative)	Many (direct, indirect, indexed, etc.)
Execution	Most instructions 1 cycle (pipelined)	Complex instructions take multiple cycles
Registers	Many general-purpose (16–32)	Fewer, often specialised
Compiler role	Critical (optimises instruction scheduling)	Less critical (hardware does more)
Examples	ARM, MIPS, RISC-V, PowerPC	x86 (Intel/AMD), VAX

WORKED EXAMPLE 9 **Choosing a Computer for a School Computer Laboratory**

A school in Douala must choose between two machines for its computer science laboratory:
Machine A: single-core CPU at 3.2 GHz, 2 KB cache, 32-bit data bus.
Machine B: dual-core CPU at 2.4 GHz, 256 KB cache, 64-bit data bus.
(a) Compute how many clock cycles per second each CPU executes per core.
(b) A program requires 1.2×10^{10} cycles on a single core. Estimate its execution time on each machine (single core used).
(c) Advise the school, justifying with at least two technical arguments.

SOLUTION

(a) Machine A: 3.2×10^9 cycles/s per core. Machine B: 2.4×10^9 cycles/s per core.
(b) Time $= \dfrac{\text{cycles}}{\text{frequency}}$.
Machine A: $t_A = \dfrac{1.2 \times 10^{10}}{3.2 \times 10^9} = 3.75$ s.
Machine B: $t_B = \dfrac{1.2 \times 10^{10}}{2.4 \times 10^9} = 5.0$ s.
For a purely single-core task, A is faster.
(c) Advice: Machine B. Justification:

- Cache: B has 256 KB against only 2 KB for A. Real programs constantly re-use data; with a larger cache, most accesses are cache hits at CPU speed, whereas A stalls waiting for RAM. In practice this can outweigh the clock advantage computed in (b), which assumed zero memory stalls.

- Two cores: a laboratory runs many simultaneous tasks (compiler, editor, browser, antivirus). Two cores execute two instruction streams in parallel, so the machine stays responsive under load.
- 64-bit data bus: B transfers twice as many bits per memory operation, improving all data-intensive work.

Conclusion: although A wins on raw single-core clock speed (3.75 s vs 5.0 s in the idealised calculation), B offers better real-world performance for a multi-tasking school environment thanks to its larger cache, second core and wider bus.

10 SECTION 2: INSTRUCTION SET ARCHITECTURE & LOW-LEVEL PROCESSING (LESSON 90)

10.1 Instruction Formats and Machine Code

DEFINITION **Instruction Format**

An **instruction format** defines the bit layout of a machine instruction. Typical fields:

- Opcode (operation code): specifies the operation (ADD, LOAD, JUMP, etc.). k bits → up to 2^k distinct instructions.
- Operand(s): specify where the data is (register number, memory address, immediate value).
- Mode bits (optional): specify the addressing mode.

Total instruction length is often a multiple of the byte (8, 16, 32, 64 bits).

METHOD **From Machine Code to Assembly and Back**

1. Identify the instruction format: how many bits for the opcode, how many for the operand(s).
2. Count the instructions: with k opcode bits, at most 2^k different instructions exist.
3. To decode a machine instruction: split the bits according to the format, look up the opcode in the instruction set table, then interpret the operand (value or address).
4. To encode an assembly instruction: find the opcode, convert opcode and operand to binary, pad to the correct number of bits, then concatenate.

5. Classify the addressing mode: immediate (operand is a value), direct (operand is an address), register (operand is a register), indirect, indexed, etc.

Reflex: always check that the total number of bits matches the instruction length given in the question.

WORKED EXAMPLE 10 **Encoding and Decoding 8-Bit Instructions**

A simple CPU uses 8-bit instructions: 3 bits of opcode followed by 5 bits of operand. Part of its instruction set is:

Opcode	Mnemonic	Meaning
000	HLT	stop
001	LOAD *n*	ACC ← Memory[*n*]
010	ADD *n*	ACC ← ACC + Memory[*n*]
011	STORE *n*	Memory[*n*] ← ACC
100	JMP *n*	PC ← *n*

(a) Encode `ADD 9` in binary. (b) Decode the instruction `01000110` if the format is 2 opcode bits + 6 operand bits with opcodes 00 = HLT, 01 = LOAD, 10 = ADD. (c) What is the largest memory address the 5-bit operand of the first format can reference?

SOLUTION

(a) Opcode of ADD = 010. Operand 9 = 01001_2 (5 bits: 9 = 8 + 1). Concatenation: `010 01001`, i.e. the machine instruction `01001001`.
(b) Split `01000110` into 2 bits + 6 bits: opcode `01`, operand `000110` = 6. Opcode 01 = LOAD, so the instruction is `LOAD 6`: ACC ← Memory[6].
(c) With 5 operand bits, the largest address is $2^5 - 1 = 31$. Only the first 32 memory locations (addresses 0 to 31) can be referenced directly.

WARNING **Common trap**

Do not confuse the number of instructions (fixed by opcode bits) with the addressable memory (fixed by operand/address bits). Here: $2^3 = 8$ possible instructions, but only $2^5 = 32$ addressable locations.

10.2 Addressing Modes

DEFINITION **Addressing Modes**

The **addressing mode** specifies how the operand field of an instruction is interpreted to obtain the actual data (effective address or value).

Mode	Description	Example (Assembly)
Immediate	Operand is the value	`MOV AX, 5`
Direct (Absolute)	Operand is the memory address	`MOV AX, [2000h]`
Register	Operand is a register	`MOV AX, BX`
Register Indirect	Register holds the address	`MOV AX, [BX]`
Indexed	Base register + offset	`MOV AX, [SI+10h]`
PC-Relative	PC + offset (for branches)	`JMP LOOP`

WORKED EXAMPLE 11 **Identifying Addressing Modes**

For each instruction, state the addressing mode of the source operand:

1. `MOV AX, 1234h`
2. `MOV AX, [1234h]`
3. `MOV AX, BX`
4. `MOV AX, [BX]`
5. `MOV AX, [BX+10h]`

SOLUTION

1. Immediate — the value 1234h is encoded directly in the instruction.
2. Direct (Absolute) — 1234h is the memory address of the operand.
3. Register — the operand is in register BX.
4. Register Indirect — BX contains the address of the operand.
5. Indexed (Base+Offset) — effective address = BX + 10h.

10.3 Assembly Language Fundamentals

DEFINITION **Assembly Language**

Assembly language is a low-level symbolic representation of machine code. Each mnemonic corresponds to one machine instruction. An **assembler** translates assembly into machine code (binary). Key elements:

- Mnemonics: e.g. `MOV`, `ADD`, `SUB`, `CMP`, `JMP`, `JE`, `JNE`, `JG`, `JL`, `CALL`, `RET`, `PUSH`, `POP`, `HLT`.
- Operands: registers, memory addresses, immediate values, labels.
- Directives (pseudo-ops): `ORG`, `DB`, `DW`, `EQU`, `END` — guide the assembler but generate no machine code.
- Comments: start with `;` — essential for readability and GCE marks.

METHOD **Assembly Programming with Registers**

1. Choose registers explicitly (e.g. AX, BX or R0, R1) and say what each one holds.
2. One instruction = one simple action: move a value (`MOV`), compute (`ADD`, `SUB`), compare (`CMP`), branch (`JMP`, `JE`, `JNE`, `JG`).
3. Translate high-level structures: an `if` becomes `CMP` + conditional jump; a `while` loop becomes a backward conditional jump to a label.
4. Trace registers in a table, one row per instruction executed, to verify the program.
5. Comment every line — at GCE, comments show understanding and earn marks.

Reflex: `CMP a, b` computes $a - b$ and sets the **flags** (Zero, Sign, Carry, Overflow); the conditional jump that follows reads those flags.

WORKED EXAMPLE 12 **Sum of the First N Integers in Assembly**

Write an assembly program (using registers AX = running total, BX = counter, CX = limit N) that computes $1 + 2 + \cdots + N$ for $N = 5$, then trace it.

SOLUTION

Program:

```
MOV CX, 5    ; CX ← N = 5

MOV AX, 0    ; AX ← total = 0

MOV BX, 1    ; BX ← counter = 1
```

```
LOOP: ADD AX, BX ; total ← total + counter
      INC BX     ; counter ← counter + 1
      CMP BX, CX ; compare counter with N
      JLE LOOP   ; repeat while counter ≤ N
HLT              ; AX now holds the sum
```

Trace table:

Instruction executed	AX	BX	CX	Comment
MOV CX, 5	0	–	5	limit set
MOV AX, 0	0	–	5	total = 0
MOV BX, 1	0	1	5	counter = 1
ADD AX, BX / INC BX	1	2	5	1 ≤ 5: loop
ADD AX, BX / INC BX	3	3	5	2 ≤ 5: loop
ADD AX, BX / INC BX	6	4	5	3 ≤ 5: loop
ADD AX, BX / INC BX	10	5	5	4 ≤ 5: loop
ADD AX, BX / INC BX	15	6	5	5 ≤ 5: loop
CMP, JLE	15	6	5	6 > 5: exit

Final result: AX = 15, which is correct since $1 + 2 + 3 + 4 + 5 = \frac{5 \times 6}{2} = 15$. The `CMP`/`JLE` pair implements the loop condition exactly as a `while (counter <= N)` in C.

10.4 Low-Level Processing — Interrupts and I/O

DEFINITION **Interrupt**

An **interrupt** is a signal (hardware or software) that causes the CPU to suspend the current program, save its state (PC, flags), and execute an **Interrupt Service Routine (ISR)** (also called interrupt handler). After the ISR, the CPU restores the state and resumes the interrupted program.

- Hardware interrupts: keyboard, mouse, timer, disk completion, network packet arrival.
- Software interrupts (exceptions): division by zero, page fault, system call (e.g. `INT 21h` in DOS, `SYSCALL` in Linux).
- Interrupt vector table: maps interrupt numbers to ISR addresses.

PROPERTY **Polling vs Interrupt-Driven I/O**

	Polling	Interrupt-Driven
Mechanism	CPU repeatedly checks device status register	Device signals CPU when ready
CPU usage	Wastes cycles waiting	CPU free for other work
Latency	Predictable, but can be high	Low latency, immediate response
Complexity	Simple	Requires interrupt controller, ISR
Typical use	Simple embedded systems, tight loops	General-purpose OS, high-speed devices

11 SECTION 3: HANDS-ON C PROJECT — CPU INSTRUCTION CYCLE & MEMORY SIMULATION (LESSON 91)

11.1 Modelling the CPU in C — Data Structures

METHOD **Writing a C Simulator of the Fetch-Decode-Execute Cycle**

1. Model memory with an array: `int memory[SIZE];` — each cell holds an encoded instruction or data.
2. Model registers with integer variables: `int PC, IR, MAR, MDR, ACC;` and a `running` flag for HLT.
3. Encode instructions as integers: e.g. instruction = opcode ×100+ operand, so `opcode = instr / 100;` and `operand = instr % 100;`.
4. Write the cycle loop: `while (running)` containing fetch (`IR = memory[PC]; PC++;`), then a `switch (opcode)` for decode-execute (LOAD, ADD, STORE, JMP, HLT).
5. Trace after each cycle: print PC, IR, ACC so the behaviour can be verified against a hand-made trace table.
6. Compile and test with a tiny program first (e.g. add two numbers), then extend.

Reflex: guard every memory access: an operand ≥ `SIZE` is a segmentation fault in C — exactly like an illegal address on real hardware.

WORKED EXAMPLE 13 A Complete CPU Simulator in C

Using the encoding instruction = 100× opcode + operand with opcodes 0 = HLT, 1 = LOAD, 2 = ADD, 3 = STORE, 4 = JMP, write a C program that loads the program "compute 7 + 5 and store the result at address 22" into memory, executes it, and prints the trace and the final value of Memory[22].

SOLUTION

Program:

```
#include <stdio.h>

#define SIZE 50

int main(void) {

  int memory[SIZE] = {0};

  int PC = 0, IR, ACC = 0, running = 1;

  int opcode, operand;

  memory[0] = 120; /* LOAD 20 */

  memory[1] = 221; /* ADD 21 */

  memory[2] = 322; /* STORE 22 */

  memory[3] = 0;   /* HLT */

  memory[20] = 7; memory[21] = 5;

  while (running) {

    IR = memory[PC]; PC++; /* fetch */

    opcode = IR / 100; operand = IR % 100; /* decode */

    switch (opcode) { /* execute */

      case 0: running = 0; break;

      case 1: ACC = memory[operand]; break;

      case 2: ACC = ACC + memory[operand]; break;

      case 3: memory[operand] = ACC; break;

      case 4: PC = operand; break;

    }

    printf("IR=%d PC=%d ACC=%d
n", IR, PC, ACC);
```

```
    }

    printf("Memory[22] = %d
n", memory[22]);

    return 0;

}
```

Output produced:

```
IR=120 PC=1 ACC=7

IR=221 PC=2 ACC=12

IR=322 PC=3 ACC=12

IR=0 PC=4 ACC=12

Memory[22] = 12
```

Explanation. The array `memory` plays the role of RAM; `PC`, `IR`, `ACC` model the registers. Each iteration of the `while` loop is one complete fetch–decode–execute cycle. The integer encoding lets us recover the opcode by integer division and the operand by the remainder. The HLT instruction (opcode 0) clears the `running` flag and stops the machine, exactly like a real halt. The final print confirms that 7 + 5 = 12 was correctly stored at address 22.

11.2 Extending the Simulator — Flags, Subroutines and Stack

EXPERIMENT **Adding a Status Register and Subroutine Support**

Extend the simulator with:

1. A `flags` variable (bit 0 = Zero, bit 1 = Carry, bit 2 = Negative, bit 3 = Overflow).
2. Instructions `CMP` (compare), `JZ` (jump if zero), `JNZ`, `JS`, `JNS`.
3. A stack in memory (growing downwards from `SIZE-1`) with a `SP` (stack pointer) register.
4. Instructions `CALL addr` (push return address, jump) and `RET` (pop return address into PC).

This mirrors real CPU behaviour and prepares for Lesson 92 assembly labs.

EXERCISE 8 **Extended Simulator Exercise**

Modify the C simulator to:

1. Add a `flags` register updated by `ADD`, `SUB`, `CMP`.
2. Implement `CMP` (sets flags like `SUB` but does not store result).
3. Implement conditional jumps `JZ`, `JNZ`, `JP` (jump if positive), `JN` (jump if negative).
4. Implement a stack with `PUSH`, `POP`, `CALL`, `RET`.
5. Write a program that calls a subroutine to compute $N!$ (factorial) for $N = 5$ using the stack for the return address.

12 SECTION 4: HANDS-ON C PROJECT — ASSEMBLY CODE & REGISTER MANIPULATION LAB (LESSON 92)

12.1 Assembly Programming — Registers, Data Movement, Arithmetic

DEFINITION **General-Purpose Registers (x86 16-bit Example)**

In 16-bit real mode (common in introductory assembly):

- AX, BX, CX, DX: 16-bit general-purpose registers. Each can be accessed as two 8-bit halves: AH/AL, BH/BL, CH/CL, DH/DL.
- SI, DI: Source Index, Destination Index — used for string/array operations.
- BP, SP: Base Pointer, Stack Pointer — used for stack frames.
- CS, DS, SS, ES: Segment registers (Code, Data, Stack, Extra).
- IP: Instruction Pointer (PC equivalent).
- FLAGS: Status register (CF, PF, AF, ZF, SF, OF, etc.).

In 32-bit protected mode: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, EIP, EFLAGS. In 64-bit long mode: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, RIP, RFLAGS, plus R8–R15.

METHOD **Translating High-Level Constructs to Assembly**

C Construct	Assembly Pattern (16-bit)
`int a = 5;`	`MOV AX, 5`
`a = b + c;`	`MOV AX, [b]\n ADD AX, [c]`
`if (a > b) {...}`	`MOV AX, [a]\n CMP AX, [b]\n JG label_then`
`while (a < 10) { a++; }`	`label_loop: MOV AX, [a] \n CMP AX, 10 \n JGE label_end\n INC AX\n MOV [a], AX\n JMP label_loop\n label_end:`
`for (i=0; i<n; i++)`	`MOV CX, 0 \n label_for: CMP CX, [n] \n JGE label_end \n ... body ... \n INC CX \n JMP label_for \n label_end:`
Function call	`CALL func\n ...\n func: ... RET`

12.2 Worked Assembly Examples with Full Traces

WORKED EXAMPLE 14 **Greatest Common Divisor (Euclidean Algorithm) in Assembly**

Write a 16-bit assembly program that computes gcd(a, b) using the Euclidean algorithm. Assume a and b are in memory locations `VAL_A` and `VAL_B`. Store the result in `RESULT`. Trace with $a = 48$, $b = 18$.

SOLUTION

Program:

```
MOV AX, [VAL_A] ; AX = a
MOV BX, [VAL_B] ; BX = b
LOOP_GCD:
  CMP BX, 0     ; while (b != 0)
  JE DONE       ; if b==0, exit
  MOV DX, 0     ; clear DX for 32-bit dividend
  DIV BX        ; AX = a/b, DX = a%b (remainder in DX)
  MOV AX, BX    ; a = b
  MOV BX, DX    ; b = remainder
  JMP LOOP_GCD
DONE:
```

```
    MOV [RESULT], AX ; store gcd
    HLT
```

Trace (a=48, b=18):

Step	AX	BX	DX	Comment
Initial	48	18	–	load a, b
CMP, JE	48	18	–	b≠0
DIV BX	2	–	12	48/18=2 rem 12
MOV AX,BX / MOV BX,DX	18	12	12	a=18, b=12
CMP, JE	18	12	–	b≠0
DIV BX	1	–	6	18/12=1 rem 6
MOV AX,BX / MOV BX,DX	12	6	6	a=12, b=6
CMP, JE	12	6	–	b≠0
DIV BX	2	–	0	12/6=2 rem 0
MOV AX,BX / MOV BX,DX	6	0	0	a=6, b=0
CMP, JE	6	0	–	b=0 → exit
MOV [RESULT], AX	6	0	–	RESULT = 6

Result: gcd(48, 18) = 6. The `DIV` instruction performs unsigned 32-bit ÷ 16-bit: dividend in DX:AX, quotient in AX, remainder in DX.

WORKED EXAMPLE 15 **Bubble Sort in Assembly — Array in Memory**

An array of 5 bytes is stored at `ARRAY: 5, 2, 8, 1, 9`. Write assembly to sort it in ascending order using bubble sort. Use registers: SI = base address, CX = outer loop counter, BX = inner loop counter, AL = temporary.

SOLUTION

Program (simplified 8-bit version for clarity):

```
  MOV SI, ARRAY   ; base address
  MOV CX, 4       ; outer loop: n-1 passes
  OUTER:
    MOV BX, 0       ; inner index = 0
    INNER:
     MOV AL, [SI+BX]   ; load arr[j]
```

```
    CMP AL, [SI+BX+1] ; compare with arr[j+1]
    JBE NO_SWAP       ; if arr[j] <= arr[j+1], no swap
    XCHG AL, [SI+BX+1] ; swap: AL <-> arr[j+1]
    MOV [SI+BX], AL   ; store swapped value
  NO_SWAP:
    INC BX            ; j++
    CMP BX, CX        ; j < n-i-1?
    JB INNER          ; if yes, continue inner
  DEC CX              ; i++
  JNZ OUTER           ; if CX!=0, continue outer
HLT
```

Trace of first pass (outer loop CX=4):

j	arr[j]	arr[j+1]	Swap?	Array after step
0	5	2	Yes	2, 5, 8, 1, 9
1	5	8	No	2, 5, 8, 1, 9
2	8	1	Yes	2, 5, 1, 8, 9
3	8	9	No	2, 5, 1, 8, 9

After 4 passes the array becomes `1, 2, 5, 8, 9`. The `XCHG` instruction atomically exchanges a register with memory — a classic assembly trick.

12.3 Interfacing Assembly with C — Inline Assembly and Calling Conventions

EXPERIMENT **Calling an Assembly Function from C (GCC, x86-64 Linux)**

On modern 64-bit Linux (System V AMD64 ABI), the first six integer/pointer arguments are passed in registers: `RDI, RSI, RDX, RCX, R8, R9`. Return value in `RAX`. The stack must be 16-byte aligned before a `CALL`.

assembly.s:

```
.global asm_gcd

asm_gcd:

  mov rax, rdi    ; a
```

```
    mov rcx, rsi    ; b
    test rcx, rcx
    je .done
.loop:
    xor rdx, rdx
    div rcx         ; rax = a/b, rdx = a%b
    mov rax, rcx    ; a = b
    mov rcx, rdx    ; b = remainder
    test rcx, rcx
    jne .loop
.done:
    ret
```

main.c:

```
#include <stdio.h>
long asm_gcd(long a, long b);
int main() { printf("gcd(48,18) = %ld
n", asm_gcd(48,18)); return 0; }
```

Compile: `gcc -o prog main.c assembly.s`. Run: `./prog` → prints `gcd(48,18) = 6`.

12.4 Debugging Assembly — Using GDB and Trace Tables

METHOD **Debugging Assembly Programs Step by Step**

1. Build with debug symbols: `nasm -f elf64 -g -F dwarf prog.asm` then `ld -o prog prog.o` (or `gcc -g prog.c asm.s`).
2. Run under GDB: `gdb ./prog`.
3. Set breakpoints: `break _start` or `break label`.
4. Step instruction by instruction: `stepi` (step into) or `nexti` (step over).
5. Examine registers: `info registers` or `i r`.

6. Examine memory: `x/10xb address` (10 hex bytes), `x/5iw address` (5 instructions).
7. Keep a paper trace table alongside GDB — write expected register values before each step, compare with `info registers`.

12.5 Practical Lab Checklist (Lesson 92)

KEY POINTS **Lab Deliverables for Lesson 92**

For the GCE practical assessment, ensure your lab report includes:

1. Problem statement and algorithm (pseudocode or flowchart).
2. Assembly source code with meaningful labels and comments on every line.
3. Register trace table for at least two test cases (including edge cases: zero, negative, maximum).
4. Screenshots of successful assembly, linking, and execution (or GDB session).
5. Analysis: compare instruction count, cycle estimate, and actual runtime with a C equivalent.
6. Reflection: what did you learn about register pressure, flag usage, stack discipline?

13 CHAPTER SUMMARY — KEY POINTS FOR GCE REVISION

KEY POINTS **Must-Know for the Examination**

1. Von Neumann architecture: single memory for instructions + data; CPU (CU, ALU, registers); three buses (address, data, control).
2. Register roles: PC (next instruction address), MAR (address bus), MDR (data bus), IR (current instruction), ACC (ALU operand/result), flags (condition codes).
3. Fetch–decode–execute cycle: fetch (PC→MAR→MDR→IR, PC++), decode (opcode/operand), execute (action). Trace tables are examinable.
4. Memory capacity: 2^n locations for n-bit address bus; byte-addressable → size in bytes = 2^n.

5. Instruction format: opcode bits → max instructions; operand bits → addressable range per instruction.
6. Addressing modes: immediate, direct, register, register indirect, indexed, PC-relative.
7. RISC vs CISC: fixed vs variable length, load/store vs memory operands, many vs few registers, compiler vs hardware complexity.
8. Performance: time = IC × CPI / clock; cache reduces effective CPI; cores give throughput, not single-thread speed.
9. C simulator: memory array, register variables, while(running) loop, fetch/decode/execute, trace printing.
10. Assembly: one mnemonic = one instruction; MOV/ADD/SUB/CMP/Jcc; loops = backward conditional jumps; subroutines = CALL/RET with stack.

EXERCISE 9 **GCE-Style Practice Questions**

1. A processor has a 24-bit address bus and a 32-bit data bus. Memory is byte-addressable. (a) How many memory locations can be addressed? (b) What is the maximum memory size in MB? (c) How many bytes are transferred per memory read cycle?
2. An 8-bit instruction format uses 4 bits for opcode and 4 bits for operand. (a) Maximum number of instructions? (b) Maximum direct address? (c) Encode `SUB 10` if opcode for SUB is 0011.
3. Trace the fetch-decode-execute cycle for the instruction `STORE 30` at address 15, given PC=15, Memory[15]=STORE 30, Memory[30]=0, ACC=42. Show PC, MAR, MDR, IR, ACC after each micro-step.
4. Write an assembly program (any syntax) to compute the sum of an array of 10 bytes starting at address `ARRAY`. Use a loop with a counter. Provide a trace table for the first two iterations assuming the array starts with `1, 2, 3, ...`.
5. Explain the difference between a hardware interrupt and a software interrupt. Give one example of each. Why is interrupt-driven I/O generally preferred over polling in a multitasking OS?

SOLUTION **Exercise 1**

(a) 2^{24} = 16 777 216 locations. (b) 16 777 216 bytes = $\frac{16\,777\,216}{1024^2}$ = 16 MB. (c) 32-bit data bus → 4 bytes transferred per cycle.

SOLUTION **Exercise 2**

(a) 2^4 = 16 instructions. (b) $2^4 - 1 = 15$ (addresses 0–15). (c) Opcode 0011, operand 10 = 1010_2 → `0011 1010` = `3A` hex.

SOLUTION **Exercise 3**

Step	PC	MAR	MDR	IR	ACC
Initial	15	–	–	–	42
Fetch	16	15	STORE 30	STORE 30	42
Execute	16	30	42	STORE 30	42

Memory[30] becomes 42.

SOLUTION **Exercise 4**

Program (NASM-style):

```
MOV SI, ARRAY
MOV CX, 10
XOR AX, AX    ; sum = 0
LOOP_SUM:
  MOV BL, [SI]   ; load byte
  ADD AL, BL    ; add to sum (8-bit)
  ADC AH, 0     ; propagate carry to AH
  INC SI        ; next element
  LOOP LOOP_SUM ; CX--, jump if CX!=0
  ; result in AX
HLT
```

Trace (first two iterations, array = 1,2,3…):

Iteration	SI	AL (sum)	CX	Memory[SI]
Start	ARRAY	0	10	1
1	ARRAY+1	1	9	2
2	ARRAY+2	3	8	3

SOLUTION **Exercise 5**

Hardware interrupt: generated by external device (e.g. keyboard press, timer tick, disk ready). Asynchronous to CPU execution. Software interrupt (exception): generated by CPU itself due to instruction (e.g. `INT 0x80` system call, division by zero, page fault). Synchronous to instruction stream. Why interrupt-driven I/O is preferred: polling wastes CPU cycles checking device status; interrupt-driven lets CPU run other processes while waiting, improving throughput and responsiveness — essential for multitasking OS.

14 EXERCISES

14.1 I check my knowledge

EXERCISE 10 **MCQ: addressing modes**

For each of the following assembly instructions, state the addressing mode used: (a) immediate, (b) direct, (c) indirect, or (d) register.

1. `MOV R1, #25`
2. `LOAD 40` (load the content of memory address 40 into the accumulator)
3. `LOAD (R2)` (load the content of the memory cell whose address is stored in register R2)
4. `ADD R1, R2`

Choose the single correct combination from the options below:

1. 1–immediate, 2–direct, 3–indirect, 4–register
2. 1–direct, 2–immediate, 3–register, 4–indirect
3. 1–immediate, 2–indirect, 3–direct, 4–register
4. 1–register, 2–direct, 3–immediate, 4–indirect

EXERCISE 11 **True or false, with justification**

Say whether each statement is true or false, and justify your answer in one or two sentences.

1. In the von Neumann architecture, instructions and data are stored in the same memory.
2. The Program Counter holds the instruction currently being executed.
3. A computer with a higher clock speed always executes programs faster than one with a lower clock speed.
4. In the instruction cycle, decoding an instruction always comes before fetching it.

EXERCISE 12 **Key definitions**

Define each of the following terms in one or two sentences, giving one example where appropriate:

1. Instruction Set Architecture (ISA)
2. Control Unit (CU)
3. Memory Address Register (MAR)
4. Opcode

EXERCISE 13 **Direct calculation: machine code fields**

An 8-bit instruction word is divided into a 3-bit **opcode** (bits 7–5) and a 5-bit **operand** (bits 4–0).

1. How many different instructions can this ISA define?
2. What is the largest memory address that the operand field can represent?
3. The machine instruction `01100101` is fetched. Give, in binary, the opcode and the operand, and the operand's value in decimal.

14.2 I practise

EXERCISE 14 **Tracing the fetch–decode–execute cycle**

A simple CPU has registers PC, IR, MAR, MDR and ACC. Memory contains: address 10 holds the value 7; address 11 holds the value 5. The program stored from address 0 is:

```
0: LOAD 10
1: ADD 11
2: STORE 12
```

Copy and complete a trace table showing the values of PC, IR, MAR, MDR and ACC after the fetch, decode and execute stages of each of the three instructions. State the final content of memory address 12.

EXERCISE 15 **Assembly to machine code**

Using the opcode table below (3-bit opcode + 5-bit operand, operand = 0 when unused), convert each instruction into 8-bit machine code, then explain the meaning of

each field.

Mnemonic	Opcode	Meaning
MOV R1, #n	001	load constant n into R1
ADD R1, R2	010	R1 ← R1 + R2
STORE n	011	store ACC at address n

1. `MOV R1, #5`
2. `ADD R1, R2`
3. `STORE 12`

EXERCISE 16 **Bit manipulation in C**

A register is modelled by `unsigned char R = 0b10110100;`. Write a single C expression for each task, and give the resulting value of `R` in binary and in hexadecimal:

1. test whether bit 2 is set (expression evaluating to non-zero if set);
2. set bit 0, leaving all other bits unchanged;
3. clear bit 7, leaving all other bits unchanged;
4. toggle (invert) bit 3.

EXERCISE 17 **Hand-tracing a countdown loop**

Trace the following assembly program step by step (initially `R1 = 0`, `R2 = 0`, zero flag Z = 0):

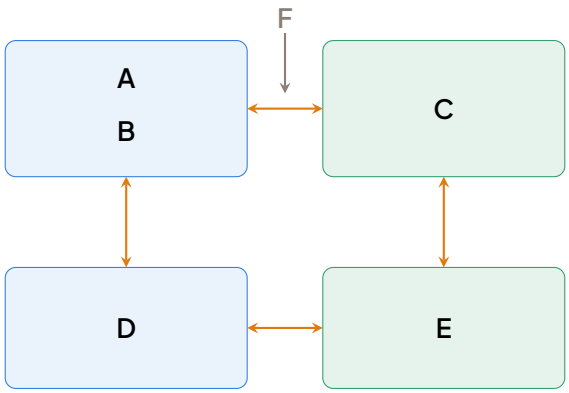
```
MOV R1, #3
LOOP: DEC R1   (R1 ← R1 -1; Z = 1 if result is 0)
INC R2   (R2 ← R2 +1)
JNZ LOOP   (jump to LOOP if Z = 0)
STORE R2, 20
```

Give the number of times the loop body executes, and the final values of `R1`, `R2`, the zero flag Z, and memory address 20.

14.3 I prepare for the GCE

EXERCISE 18 **Labelling the von Neumann architecture** **(10 marks)**

The diagram below shows the von Neumann architecture with its labels replaced by the letters A–F.



Von Neumann architecture with hidden labels

1. Name the components A, B, C, D and E, given that A and B are the two parts of the CPU. (5 marks)
2. State the function of each of the components A, C and D. (3 marks)
3. What does the set of connections F represent? Name two buses it contains and state the role of each. (2 marks)

EXERCISE 19 **CPU and memory simulation in C** **(12 marks)**

A student simulates a tiny 8-bit computer in C with the declarations:

```
unsigned char memory[256];

unsigned char ACC = 0, PC = 0, IR, MAR, MDR;
```

The opcodes are: `LOAD = 1` (ACC ← memory[operand]), `ADD = 2` (ACC ← ACC + memory[operand]), `STORE = 3` (memory[operand] ← ACC).

1. Write a C function `unsigned char fetch(void)` that copies the instruction at address `PC` into `IR` and increments `PC`. (3 marks)
2. Write a C function `void execute(unsigned char opcode, unsigned char operand)` that implements LOAD, ADD and STORE using a `switch` statement. (5 marks)
3. Explain why `unsigned char` is an appropriate type for simulating an 8-bit register, and state what happens in your simulation when `ADD` produces a result greater than 255. (2 marks)

4. The simulated memory is limited to 256 cells. Explain which register in a real CPU determines the maximum addressable memory, and how. (2 marks)

EXERCISE 20 **Essay: same clock speed, different performance** **(15 marks)**

Two computers, *X* and *Y*, have processors running at exactly the same clock frequency of 3.0 GHz, yet a given program runs noticeably faster on *X* than on *Y*.
Write a structured essay (introduction, development, conclusion) explaining why clock speed alone does not determine performance. Your answer must refer to at least the following points, with a clear technical explanation of each:

1. the role of **cache memory** and the cost of accessing main memory;
2. the influence of **word size** and data bus width;
3. architectural differences: number of cores, pipelining, and the number of clock cycles per instruction (CPI) of the instruction set;
4. one other relevant hardware factor of your choice (e.g. RAM size, secondary storage speed).

Illustrate your essay with a simple numerical example comparing the execution times of the two machines for a program of 10^9 instructions with different average CPI values.

15 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: addressing modes**

The correct answer is option A: 1–immediate, 2–direct, 3–indirect, 4–register.
Justification:

1. `MOV R1, #25`: the value 25 itself is written inside the instruction (the symbol # marks a constant). The operand is the data: this is immediate addressing.
2. `LOAD 40`: the instruction contains the address (40) of the memory cell holding the data: this is direct addressing.
3. `LOAD (R2)`: the instruction names a register R2 which contains the address of the data in memory; one must first read R2, then access memory: this is indirect (register indirect) addressing.
4. `ADD R1, R2`: both operands are held in registers, no memory access is needed for the data: this is register addressing.

SOLUTION **Exercise 2 — True or false, with justification**

1. True. In the von Neumann (stored-program) architecture, instructions and data share the same memory and travel on the same buses; they are distinguished only by the way the CPU interprets them.
2. False. The Program Counter holds the address of the next instruction to be fetched. The instruction currently being executed is held in the Instruction Register (IR).
3. False. Performance also depends on the number of cycles per instruction (CPI), cache memory, word size, number of cores, pipelining, etc. A slower-clocked machine with a better architecture can outperform a faster-clocked one.
4. False. The order is always fetch → decode → execute: an instruction must first be brought from memory into the IR before the Control Unit can decode it.

SOLUTION **Exercise 3 — Key definitions**

1. Instruction Set Architecture (ISA): the complete specification of the instructions a processor can execute — the set of opcodes, the registers, the addressing modes and the binary format of instructions. Example: the ISA defines that `ADD R1, R2` adds

two registers.

2. Control Unit (CU): the part of the CPU that directs operations: it fetches and decodes each instruction and sends control signals to the ALU, registers and memory so that the instruction is executed in the correct order.
3. Memory Address Register (MAR): a CPU register that holds the address of the memory location to be read from or written to; it is connected to the address bus. Example: to execute `LOAD 10`, the value 10 is placed in the MAR.
4. Opcode: the part of a machine instruction that codes the operation to perform (as opposed to the operand, which says on what). Example: in `01100101`, the 3-bit opcode `011` might mean STORE.

SOLUTION **Exercise 4 — Direct calculation: machine code fields**

1. The opcode has 3 bits, so the ISA can define $2^3 = 8$ different instructions (opcodes `000` to `111`).
2. The operand has 5 bits, so the largest address is $2^5 - 1 = 31$ (binary `11111`).
3. Split `01100101` into bits 7–5 and bits 4–0:
   - opcode = `011` (value 3 in decimal);
   - operand = `00101`;
   - operand value in decimal: $0 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 5$.

SOLUTION **Exercise 5 — Tracing the fetch–decode–execute cycle**

Recall the roles: during fetch, MAR ← PC, MDR ← memory[MAR], IR ← MDR, PC ← PC +1. During execute of `LOAD n` / `ADD n`, MAR ← *n* and MDR ← memory[n].
Instruction 1: `LOAD 10`

Stage	PC	IR	MAR	MDR	ACC
Fetch	1	LOAD 10	0	LOAD 10	–
Decode	1	LOAD 10	0	LOAD 10	–
Execute	1	LOAD 10	10	7	7

Instruction 2: `ADD 11`

Stage	PC	IR	MAR	MDR	ACC
Fetch	2	ADD 11	1	ADD 11	7
Decode	2	ADD 11	1	ADD 11	7
Execute	2	ADD 11	11	5	12

Instruction 3: `STORE 12`

Stage	PC	IR	MAR	MDR	ACC
Fetch	3	STORE 12	2	STORE 12	12
Decode	3	STORE 12	2	STORE 12	12
Execute	3	STORE 12	12	12	12

At execute of `STORE 12`, the ACC value is copied via the MDR into memory: memory[12] ← 12.
Final content of memory address 12: **12** (which is indeed 7 + 5).

SOLUTION **Exercise 6 — Assembly to machine code**

Format: bits 7–5 = opcode (3 bits), bits 4–0 = operand (5 bits).

1. `MOV R1, #5`: opcode `001`, operand 5 = `00101`. Machine code: `00100101`. Meaning: opcode `001` = load a constant into R1; operand `00101` = the constant 5.
2. `ADD R1, R2`: opcode `010`, operand unused = `00000`. Machine code: `01000000`. Meaning: opcode `010` = R1 ← R1 + R2; operand field is `00000` because the instruction works only on registers.
3. `STORE 12`: opcode `011`, operand 12 = 8 + 4 = `01100`. Machine code: `01101100`. Meaning: opcode `011` = store ACC in memory; operand `01100` = the destination address 12.

Instruction	Opcode	Operand	Machine code
MOV R1, #5	001	00101	00100101
ADD R1, R2	010	00000	01000000
STORE 12	011	01100	01101100

SOLUTION **Exercise 7 — Bit manipulation in C**

Initial value: `R = 0b10110100` = 180 = `0xB4`. Bits are numbered from 0 (rightmost) to 7 (leftmost).

1. Test bit 2: `R & (1 << 2)`. Mask `00000100`; bit 2 of `10110100` is 1, so the expression eval-

uates to `00000100` = 4 (non-zero ⇒ bit 2 is set). R is unchanged: `0b10110100` = `0xB4`.

2. Set bit 0: `R = R | (1 << 0)` (or `R |= 1`). Result: `0b10110101` = `0xB5`.
3. Clear bit 7: `R = R & ~(1 << 7)` (or `R &= 0x7F`). Mask `01111111`; result: `0b00110100` = `0x34`.
4. Toggle bit 3: `R = R ^ (1 << 3)` (or `R ^= 8`). Bit 3 of `10110100` is 1, so it becomes 0: `0b10111100` → wait — flipping bit 3 of `10110100`: bits are 1 0 1 1 0 1 0 0 (bit 7 to bit 0); bit 3 = 0, so it becomes 1. Result: `0b10111100` = `0xBC`.

WARNING **Check each bit position carefully**

Always write out the 8 bits and number them 7 down to 0 before applying a mask. Here bit 2 = 1 and bit 3 = 0 in the original value `10110100`; confusing the two positions is the most common error.

SOLUTION **Exercise 8 — Hand-tracing a countdown loop**

Trace table (state after each instruction):

Instruction executed	R1	R2	Z
initial state	0	0	0
MOV R1, #3	3	0	0
DEC R1 (pass 1)	2	0	0
INC R2	2	1	0
JNZ LOOP (Z=0 ⇒ jump)	2	1	0
DEC R1 (pass 2)	1	1	0
INC R2	1	2	0
JNZ LOOP (Z=0 ⇒ jump)	1	2	0
DEC R1 (pass 3)	0	2	1
INC R2	0	3	1
JNZ LOOP (Z=1 ⇒ no jump)	0	3	1
STORE R2, 20	0	3	1

- The loop body (`DEC`, `INC`, `JNZ`) executes 3 times.
- Final values: `R1 = 0`, `R2 = 3`, zero flag `Z = 1`.
- Memory address 20 receives the value of R2: memory[20] = 3.

The program simply counts down R1 from 3 to 0 while counting up R2, so R2 ends with the initial value of R1.

SOLUTION **Exercise 9 — Labelling the von Neumann architecture (10 marks)**

1. Naming the components (5 marks — 1 mark each):

- A = Control Unit (CU) and B = Arithmetic and Logic Unit (ALU) — together they form the CPU (box A–B). (Accept A and B interchanged only if the functions in part 2 are consistent; conventionally the CU is listed first.)
- C = Main memory (RAM) — connected to the CPU by buses.
- D = Input unit (keyboard, mouse, scanner…).
- E = Output unit (screen, printer, speakers…).

2. Functions (3 marks — 1 mark each):

- A (Control Unit): fetches, decodes and sequences instructions; sends control signals coordinating the ALU, registers, memory and I/O.
- C (Main memory): stores both the program instructions and the data being processed, in addressable cells.
- D (Input unit): converts data and programs from the outside world into binary form and transfers them into memory/the CPU.

3. The connections F (2 marks): F represents the system bus linking the CPU, memory and I/O. It contains, among others:

- the address bus: carries the address of the memory cell (or port) to access, from the CPU (MAR) to memory — 1 mark;
- the data bus: carries the actual data/instructions between memory and the CPU (MDR), in both directions — 1 mark.

(The control bus, carrying read/write and timing signals, is also accepted as one of the two.)
Examiner expectations: precise vocabulary (CU, ALU, MAR not required here), functions stated as actions ("fetches and decodes", not just "controls"), and for F the direction of each bus must be indicated.

SOLUTION **Exercise 10 — CPU and memory simulation in C (12 marks)**

1. The fetch function (3 marks):

```
unsigned char fetch(void) {
```

```
  MAR = PC;
  MDR = memory[MAR];
  IR = MDR;
  PC = PC + 1;
  return IR;
}
```

Marking: 1 mark for copying memory[PC] into IR (via MAR/MDR acceptable or direct), 1 mark for incrementing PC, 1 mark for correct signature and return.

2. The execute function (5 marks):

```
void execute(unsigned char opcode, unsigned char operand) {
  switch (opcode) {
    case 1: /* LOAD */
      MAR = operand;
      MDR = memory[MAR];
      ACC = MDR;
      break;
    case 2: /* ADD */
      MAR = operand;
      MDR = memory[MAR];
      ACC = ACC + MDR;
      break;
    case 3: /* STORE */
      MAR = operand;
      MDR = ACC;
      memory[MAR] = MDR;
      break;
  }
}
```

Marking: 1 mark per correct case (LOAD, ADD, STORE), 1 mark for the `switch` structure with `break`s, 1 mark for coherent use of MAR/MDR.
3. Why `unsigned char` (2 marks): An `unsigned char` occupies exactly 8 bits and holds values 0–255, matching an 8-bit register. When `ADD` gives a result greater than 255, the value wraps around (modulo 256): e.g. 200 + 100 = 300 is stored as 300 − 256 = 44. This imitates real 8-bit overflow, where the excess bit is lost and a carry/overflow flag would be set in a real CPU (1 mark for the type justification, 1 mark for wrap-around/modulo 256).
4. Maximum addressable memory (2 marks): It is the MAR (Memory Address Register) — equivalently the width of the address bus — that limits addressable memory: with an n-bit MAR, at most 2^n distinct addresses exist. Here the 8-bit registers give $2^8 = 256$ cells, exactly the size of the simulated `memory[256]`.

SOLUTION **Exercise 11 — Essay: same clock speed, different performance (15 marks)**

Indicative mark scheme: introduction 1 mark; cache/memory hierarchy 3 marks; word size and bus width 3 marks; architectural differences (cores, pipelining, CPI) 4 marks; one other factor 2 marks; numerical example 1 mark; conclusion and quality of structure/expression 1 mark.
Model essay.
Introduction. Clock frequency only tells us how many clock cycles occur per second. The time to run a program is given by

$$T = \frac{\text{number of instructions} \times \text{CPI}}{\text{clock frequency}},$$

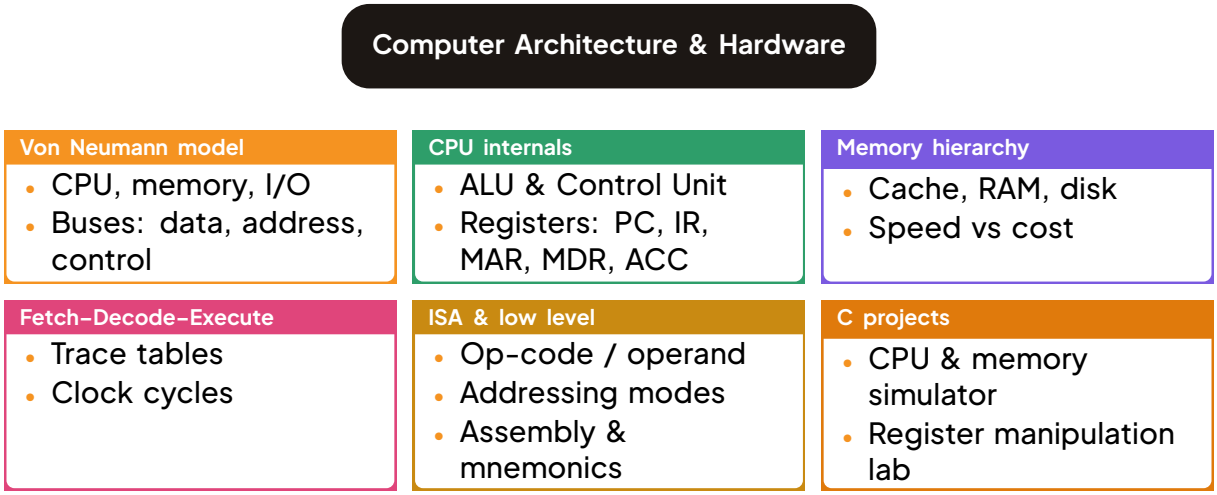
so two machines at the same 3.0 GHz can give very different execution times if their cycles-per-instruction (CPI) or effective instruction counts differ.
Cache memory. Main memory (RAM) is much slower than the CPU: an access may cost tens of cycles. Cache is a small, fast memory inside the processor holding recently used instructions and data. If machine X has a larger or better-organised cache, most of its accesses are cache hits served in one or two cycles, while Y suffers frequent cache misses and must wait for RAM. Its average CPI therefore rises, and the program slows down even at identical frequency.
Word size and data bus width. The word size is the number of bits the CPU processes in one operation. A 64-bit machine with a 64-bit data bus can fetch and add 8 bytes at once; a 32-bit machine needs two operations for the same data. For data-intensive programs, X thus executes fewer instructions and fewer memory transfers than Y at the same clock rate.
Architectural differences. First, cores: if X has several cores and the program is parallelisable, work is shared between cores while Y's single core does everything sequentially. Second, pipelining: a deeper, well-designed pipeline overlaps fetch, decode and execute of several instructions, approaching one instruction completed per cycle; stalls (branches, hazards) are better handled on X. Third, the ISA and CPI: a richer instruction set may perform in one instruction what costs Y three or four; even for the same program, the

average CPI differs between designs.
Numerical example. Take a program of 10^9 instructions at $f = 3.0\ \text{GHz} = 3 \times 10^9$ cycles/s. If X achieves an average CPI of 1 and Y an average CPI of 2.5 (cache misses, no pipelining benefit):

$$T_X = \frac{10^9 \times 1}{3 \times 10^9} \approx 0.33\ \text{s}, \qquad T_Y = \frac{10^9 \times 2.5}{3 \times 10^9} \approx 0.83\ \text{s}.$$

Same clock speed, yet X is about 2.5 times faster.
Another factor: RAM size and storage speed. If Y has too little RAM, the operating system swaps pages to the hard disk or SSD, which is orders of magnitude slower than RAM; a fast SSD on X reduces loading and swapping time. Either effect can dominate the running time of real applications.
Conclusion. Clock frequency is only one factor of performance. Cache behaviour, word size and bus width, the number of cores, pipelining, the CPI of the instruction set, and the memory/storage hierarchy all determine how much useful work is done per cycle — which is why X outperforms Y at the same 3.0 GHz.
Examiner expectations: a structured essay (visible introduction/development/conclusion), each point explained technically (not merely named), the formula $T = \frac{N \times \text{CPI}}{f}$ used correctly, coherent units, and a numerical comparison that is arithmetically correct.

16 SUMMARY SHEET



Mind map of Chapter CSC16: from the Von Neumann model to hands-on C simulation.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- **Computer architecture**: the functional organisation of a computer (CPU, memory, input/output, buses) as seen by the programmer.
- **Von Neumann architecture**: a single memory stores both instructions and data; instructions are fetched and executed sequentially.
- **CPU** (Central Processing Unit): made of the **Control Unit** (decodes and coordinates), the **ALU** (arithmetic and logic operations) and **registers** (very fast internal storage).
- **Registers to know**: PC (Program Counter: address of next instruction), IR (Instruction Register: current instruction), MAR (Memory Address Register), MDR (Memory Data Register), ACC (Accumulator: results of the ALU).
- **Buses**: data bus (carries data, bidirectional), address bus (carries addresses, unidirectional CPU → memory), control bus (read/write signals, interrupts).
- **Memory hierarchy**: registers > cache > RAM > secondary storage; going down: cheaper per byte but slower and larger.
- **ISA** (Instruction Set Architecture): the set of machine instructions a processor understands; each instruction = op-code (what to do) + operand(s) (on what).
- **Addressing modes**: immediate (value given), direct (address given), indirect (address of the address), register.

- **Assembly language**: human-readable mnemonics (LOAD, STORE, ADD, SUB, JMP, JZ, HALT) translated to machine code by an assembler.

Laws / formulas / facts to know

- The **fetch–decode–execute cycle**: (1) MAR ← PC; (2) MDR ← Memory[MAR], IR ← MDR, PC ← PC + 1; (3) CU decodes IR; (4) instruction executed (operands fetched, ALU used, result stored); repeat.
- An address bus of n bits addresses 2^n memory locations; a data bus of m bits transfers m bits per cycle.
- Machine cycle time = $\frac{1}{\text{clock frequency}}$; e.g. a 2 GHz clock gives a 0.5 ns cycle.
- Little Man Computer (LMC) style instructions: 3-digit code = 1 digit op-code + 2 digits operand address.

Methods

- Tracing a program: build a trace table with one column per register (PC, IR, MAR, MDR, ACC) and per memory cell used; update one row per cycle.
- Assembly → machine code: identify the op-code of each mnemonic, attach the operand address, assemble line by line.
- Simulating in C: represent memory with an array (`int memory[100];`), registers with variables, and the cycle with a `while` loop containing `fetch`, `decode` (`switch` on op-code) and `execute`.
- Register lab in C: use bitwise operators (`&`, `|`, `^`, `<<`, `>>`) to set, clear and test individual bits of a register variable.

Common mistakes

- Confusing the address bus (unidirectional) with the data bus (bidirectional).
- Forgetting to increment the PC during fetch — the program then loops on one instruction.
- Mixing up MAR (holds an address) and MDR (holds data).
- Believing cache is bigger than RAM: it is faster but smaller.
- In C, reading an uninitialised memory cell or register — always initialise the simulator's memory.
- Confusing op-code and operand when hand-assembling.

GCE tips

- Always draw the Von Neumann diagram (CPU – buses – memory – I/O) when asked to describe architecture: a labelled diagram earns marks.
- In trace questions, show every register change step by step; examiners reward the method, not just the final ACC value.

- Quote the three buses by name and direction; state clearly that instructions and data share the same memory (Von Neumann).
- For the C project questions, comment your code and keep the fetch–decode–execute structure visible with a `switch` statement.
- Manage time: architecture questions are often short-answer; be precise and use the exact register names (PC, IR, MAR, MDR, ACC).

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+